

Physlib Monthly Report

1–31 May 2025

Contributors: Eric Wieser, Joseph Tooby-Smith, Matteo Cipollina, Pietro Monticone, Zhi Kai Pong

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit `e41a3c8`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 May 2025 on branch `master` of [leanprover-community/physlib](#), between commits `a30f2f3` and `e41a3c8`. Over that period 5 contributors landed 99 commits, changing 11,817 lines across 183 files and adding 415 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.Space.VectorIdentities</code>	3
2.2	<code>PhysLean.ClassicalMechanics.Time.Basic</code>	4
2.3	<code>PhysLean.ClassicalMechanics.VectorFields</code>	4
2.4	<code>PhysLean.ClassicalMechanics.WaveEquation.Basic</code>	5
2.5	<code>PhysLean.CondensedMatter.TightBindingChain.Basic</code>	7
2.6	<code>PhysLean.Electromagnetism.Wave</code>	10
2.7	<code>PhysLean.Mathematics.FDerivCurry</code>	13
2.8	<code>PhysLean.Mathematics.Geometry.Metric.PseudoRiemannian.Defs</code>	13
2.9	<code>PhysLean.Mathematics.Geometry.Metric.Riemannian.Defs</code>	21
2.10	<code>PhysLean.QuantumMechanics.FiniteTarget.HilbertSpace</code>	23
2.11	<code>PhysLean.Relativity.PauliMatrices.CliffordAlgebra</code>	24
2.12	<code>PhysLean.Relativity.SpaceTime.CliffordAlgebra</code>	24
2.13	<code>PhysLean.Relativity.Tensors.Color.Basic</code>	25
2.14	<code>PhysLean.Relativity.Tensors.Color.Lift</code>	26
2.15	<code>PhysLean.StringTheory.FTheory.SU5U1.AnomalyCancellation.Basic</code>	29
2.16	<code>PhysLean.StringTheory.FTheory.SU5U1.Charges.Basic</code>	29
2.17	<code>PhysLean.StringTheory.FTheory.SU5U1.Charges.OfRationalSection</code>	36
2.18	<code>PhysLean.StringTheory.FTheory.SU5U1.Fluxes.Basic</code>	38
2.19	<code>PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.ChiralIndices</code>	41

2.20	<code>PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.Completeness</code>	44
2.21	<code>PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.Elems</code>	48
2.22	<code>PhysLean.StringTheory.FTheory.SU5U1.Matter</code>	49
2.23	<code>PhysLean.StringTheory.FTheory.SU5U1.Potential.Basic</code>	51
2.24	<code>PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Basic</code>	52
2.25	<code>PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Basic</code>	57
2.26	<code>PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Completeness</code>	58
2.27	<code>PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Elems</code>	59

1 Introduction

During May 2025, 5 contributors submitted 99 commits that changed 11,817 lines (+9,238 / -2,579) across 183 files spanning 20 top-level areas of the repository. Of those, 20 files were newly added and 6 were removed outright. This report catalogues 415 newly added Lean declarations: 255 lemmas, 100 defs, 35 instances, 11 abbrevs, 8 theorems, 3 structures, 2 semiformal results, and 1 inductive. We're delighted to recognize the following first-time contributor this month: [Eric Wieser](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Eric Wieser	314	5
Joseph Tooby-Smith	21,588	31
Matteo Cipollina	1,561	17
Pietro Monticone	670	10
Zhi Kai Pong	2,658	36

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	16

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/StringTheory	19	+12	-5	+4,823	-1,040
PhysLean/Relativity	48	+1	—	+689	-596
PhysLean/Mathematics	12	+2	—	+932	-235
scripts	2	+1	—	+968	-0
PhysLean/Electromagnetism	4	+1	—	+599	-26
PhysLean/ClassicalMechanics	7	+1	—	+464	-26
PhysLean/QFT	48	—	—	+146	-264
PhysLean/CondensedMatter	1	—	—	+346	-4
PhysLean/QuantumMechanics	7	+1	—	+104	-110
docs/_data	1	—	-1	+0	-190
PhysLean/Particles	18	—	—	+32	-47
.github/workflows	2	+1	—	+58	-2
lake-manifest.json	1	—	—	+18	-18
PhysLean.lean	1	—	—	+18	-5
README.md	1	—	—	+16	-3
scripts/MetaPrograms	2	—	—	+15	-3

PhysLean/Meta	6	—	—	+6	-6
lakefile.toml	1	—	—	+2	-2
PhysLean/Cosmology	1	—	—	+1	-1
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	255
def	100
instance	35
abbrev	11
theorem	8
structure	3
semiformal_result	2
inductive	1

2 Details by file

2.1 PhysLean.ClassicalMechanics.Space.VectorIdentities

[PhysLean/ClassicalMechanics/Space/VectorIdentities.lean](#) on GitHub.

lemma grad_zero [\[source\]](#)

```
@[simp]
lemma grad_zero : ∀ (θ : Space d → ℝ) = 0
```

lemma div_zero [\[source\]](#)

```
@[simp]
lemma div_zero : ∀ · (θ : Space d → EuclideanSpace ℝ (Fin d)) = 0
```

lemma curl_zero [\[source\]](#)

```
@[simp]
lemma curl_zero : ∀ × (θ : Space → EuclideanSpace ℝ (Fin 3)) = 0
```

lemma grad_const [\[source\]](#)

```
@[simp]
lemma grad_const : ∀ (fun _ : Space d => c) = 0
```

lemma div_const [\[source\]](#)

```
@[simp]
lemma div_const : ∀ · (fun _ : Space d => v) = 0
```

lemma curl_const[\[source\]](#)

```
@[simp]
lemma curl_const :  $\nabla \times (\text{fun } _ : \text{Space} \Rightarrow v_3) = 0$ 
```

2.2 PhysLean.ClassicalMechanics.Time.Basic

[PhysLean/ClassicalMechanics/Time/Basic.lean](#) on GitHub.

lemma deriv_smul[\[source\]](#)

```
lemma deriv_smul (f : Time → EuclideanSpace ℝ (Fin d)) (k : ℝ)
  (hf : Differentiable ℝ f) :
   $\partial_t (\text{fun } t \Rightarrow k \cdot f\ t) = k \cdot \partial_t (\text{fun } t \Rightarrow f\ t)$ 
```

2.3 PhysLean.ClassicalMechanics.VectorFields

[PhysLean/ClassicalMechanics/VectorFields.lean](#) on GitHub.

lemma fderiv_cross_commute[\[source\]](#)

Cross product and fderiv commute.

```
lemma fderiv_cross_commute {u : ℝ} {s : Space} {f : ℝ → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) :
   $s \times_{e_3} (fderiv\ \mathbb{R}\ (\text{fun } u \Rightarrow f\ u)\ u) =$ 
   $fderiv\ \mathbb{R}\ (\text{fun } u \Rightarrow s \times_{e_3} (f\ u))\ u$ 
```

lemma time_deriv_cross_commute[\[source\]](#)

Cross product and time derivative commute.

```
lemma time_deriv_cross_commute {s : Space} {f : Time → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) :
   $s \times_{e_3} (\partial_t (\text{fun } t \Rightarrow f\ t)) =$ 
   $\partial_t (s \times_{e_3} (f\ t))$ 
```

def rec[\[source\]](#)

Pending #25193.

```
@[elab_as_elim, cases_eliminator]
def _root_.WithLp.rec {V} {p} (motive : WithLp p V → Sort*)
  (toLp :  $\forall v, \text{motive } ((\text{WithLp.equiv } p \_).symm\ v)$ ) :  $\forall v, \text{motive } v$ 
```

lemma inner_cross_self[\[source\]](#)

```
lemma inner_cross_self (v w : EuclideanSpace ℝ (Fin 3)) :
   $\text{inner}\ \mathbb{R}\ v\ (w \times_{e_3} v) = 0$ 
```

lemma inner_self_cross[\[source\]](#)

```
lemma inner_self_cross (v w : EuclideanSpace ℝ (Fin 3)) :
  inner ℝ v (v ×e3 w) = 0
```

2.4 PhysLean.ClassicalMechanics.WaveEquation.Basic

[PhysLean/ClassicalMechanics/WaveEquation/Basic.lean](#) on GitHub.

def WaveEquation[\[source\]](#)

The general form of the wave equation where c is the propagation speed.

```
def WaveEquation (f : Time → Space d → EuclideanSpace ℝ (Fin d))
  (t : Time) (x : Space d) (c : ℝ) : Prop
```

def planeWave[\[source\]](#)

A vector-valued plane wave travelling in the direction of s with propagation speed c .

```
@[nolint unusedArguments]
noncomputable def planeWave (f₀ : ℝ → EuclideanSpace ℝ (Fin d))
  (c : ℝ) (s : Space d) (hs : inner ℝ s s = 1) : Time → Space d → EuclideanSpace ℝ (Fin d)
```

lemma wave_dt[\[source\]](#)

```
lemma wave_dt {s : Space d} {f₀' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  (h' : ∀ x, HasFDerivAt f₀ (f₀' x) x) :
  ∂t (fun t => f₀ (inner ℝ x s - c * t)) = fun t => -c • (f₀' (inner ℝ x s - c * t)) 1
```

lemma wave_dt2[\[source\]](#)

```
lemma wave_dt2 {s : Space d} {f₀' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  {f₀'' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  (h'' : ∀ x, HasFDerivAt (fun x' => f₀' x' 1) (f₀'' x) x) :
  ∂t (fun t => -c • (f₀' (inner ℝ x s - c * t)) 1) t
  =
  c^2 • (f₀'' (inner ℝ x s - c * t)) 1
```

lemma wave_differentiable[\[source\]](#)

```
lemma wave_differentiable {c : ℝ} {x : EuclideanSpace ℝ (Fin d)} :
  DifferentiableAt ℝ (fun x => inner ℝ x s - c * t) x
```

lemma wave_dx[\[source\]](#)

```
lemma wave_dx {u v : Fin d} {f₀' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  (h' : ∀ x, HasFDerivAt f₀ (f₀' x) x) :
  (fun x' => fderiv ℝ (fun x => (inner ℝ (f₀ (inner ℝ x s - c * t))
    (EuclideanSpace.single u 1))) x' (EuclideanSpace.single v 1))
  =
  fun x' => (inner ℝ (f₀' (inner ℝ x' s - c * t) (s v)) (EuclideanSpace.single u 1))
```

lemma wave_dx2[\[source\]](#)

```

lemma wave_dx2 {u v : Fin d} {f₀' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  {f₀'' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d)}
  (h' : ∀ x, HasFDerivAt (fun x' => f₀' x' 1) (f₀'' x) x) :
  (fderiv ℝ (fun x' => (inner ℝ ((f₀' (inner ℝ x' s - c * t)) (s u))
    (EuclideanSpace.single v 1))) x) (EuclideanSpace.single u 1)
  =
  inner ℝ ((s u) ^ 2 • f₀'' (inner ℝ x s - c * t) 1) (EuclideanSpace.single v 1)

```

theorem planeWave_isWave[\[source\]](#)

The plane wave satisfies the wave equation.

```

theorem planeWave_isWave (c : ℝ) (s : Space d) (hs : inner ℝ s s = 1)
  (f₀ : ℝ → EuclideanSpace ℝ (Fin d)) (f₀' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d))
  (f₀'' : ℝ → ℝ → L[ℝ] EuclideanSpace ℝ (Fin d))
  (h' : ∀ x, HasFDerivAt f₀ (f₀' x) x) (h'' : ∀ x, HasFDerivAt (fun x' => f₀' x' 1) (f₀'' x)
  x) :
  ∀ t x, WaveEquation (planeWave f₀ c s hs) t x c

```

lemma space_fderiv_of_inner_product_wave_eq_space_fderiv[\[source\]](#)

*If f_0 is a function of $(\text{inner } \mathbb{R} \ x \ s - c * t)$, the $fderiv$ of its components with respect to spatial coordinates is equal to the corresponding component of the propagation direction s times time derivative.*

```

lemma space_fderiv_of_inner_product_wave_eq_space_fderiv {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {u v : Fin 3} (h' : Differentiable ℝ f₀) :
  c * ((fun x' => (fderiv ℝ (fun x => inner ℝ (f₀ (inner ℝ x s - c * t))
    (EuclideanSpace.single v 1)) x') (EuclideanSpace.single u 1)) x)
  =
  - s u * ∂ₜ (fun t => f₀ (inner ℝ x s - c * t)) t v

```

lemma time_differentiable_of_eq_planewave[\[source\]](#)

```

lemma time_differentiable_of_eq_planewave {s : Space} {hs : inner ℝ s s = 1}
  {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {f : Time → Space → EuclideanSpace ℝ (Fin 3)}
  (h' : Differentiable ℝ f₀) (hf : f = planeWave f₀ c s hs) :
  Differentiable ℝ fun t => f t x

```

lemma crossProduct_time_differentiable_of_right_eq_planewave[\[source\]](#)

```

lemma crossProduct_time_differentiable_of_right_eq_planewave {s : Space} {hs : inner ℝ s s = 1}
  {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {f : Time → Space → EuclideanSpace ℝ (Fin 3)}
  (h' : Differentiable ℝ f₀) (hf : f = planeWave f₀ c s hs) :
  DifferentiableAt ℝ (fun t => s ×ₑ₃ (f t x)) t

```

lemma crossProduct_differentiable_of_right_eq_planewave[\[source\]](#)

```

lemma crossProduct_differentiable_of_right_eq_planewave {s : Space}
  {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)} (h' : Differentiable ℝ f₀) :
  DifferentiableAt ℝ (fun u => s ×ₑ₃ (f₀ u)) u

```

lemma wave_fderiv_inner_eq_inner_fderiv_proj[\[source\]](#)

```

lemma wave_fderiv_inner_eq_inner_fderiv_proj {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {i : Fin 3} (h' : Differentiable ℝ f₀) :
  ∀ x y, s i * (fderiv ℝ (fun x => f₀ (inner ℝ x s - c * t)) x) y i
  =
  inner ℝ y s * (fderiv ℝ (fun x => f₀ (inner ℝ x s - c * t) i) x)
  (EuclideanSpace.single i 1)

```

lemma differentiable_uncurry_of_eq_planewave[\[source\]](#)

```

lemma differentiable_uncurry_of_eq_planewave {s : Space} {hs : inner ℝ s s = 1}
  {f₀ : ℝ → EuclideanSpace ℝ (Fin 3)} (hf : f = planeWave f₀ c s hs)
  (h' : Differentiable ℝ f₀) : Differentiable ℝ hf

```

2.5 PhysLean.CondensedMatter.TightBindingChain.Basic

[PhysLean/CondensedMatter/TightBindingChain/Basic.lean](#) on GitHub.

structure TightBindingChain[\[source\]](#)

The physical parameters making up the tight binding chain.

```

structure TightBindingChain where
  N : Nat
  [N_ne_zero : NeZero N]
  a : ℝ
  a_pos : 0 < a
  E0 : ℝ
  t : ℝ

```

abbrev HilbertSpace[\[source\]](#)

The Hilbert space of a TightBindingchain is the N-dimensional finite dimensional Hilbert space.

```

abbrev HilbertSpace

```

instance NeZero T.N[\[source\]](#)

```

instance : NeZero T.N

```

def localizedState[\[source\]](#)

The eigenstate corresponding to the particle been located on the nth site.

```

noncomputable def localizedState {T : TightBindingChain} :
  OrthonormalBasis (Fin T.N) ℂ (HilbertSpace T)

```

lemma localizedState_orthonormal[\[source\]](#)

The localized states are normalized.

```

lemma localizedState_orthonormal : Orthonormal ℂ (localizedState (T := T))

```


lemma localizedState_orthonormal_eq_ite[\[source\]](#)

```
lemma localizedState_orthonormal_eq_ite (m n : Fin T.N) :
  ⟨m|n⟩ = if m = n then 1 else 0
```

def localizedComp[\[source\]](#)

The linear map $|m\rangle\langle n|$ for $\langle n|$ localized states.

```
noncomputable def localizedComp {T : TightBindingChain} (m n : Fin T.N) :
  T.HilbertSpace →l[ℂ] T.HilbertSpace where
```

lemma localizedComp_apply_localizedState[\[source\]](#)

```
lemma localizedComp_apply_localizedState (m n p : Fin T.N) :
  |m⟩⟨n| |p⟩ = if n = p then |m⟩ else 0
```

def hamiltonian[\[source\]](#)

The Hamiltonian of the tight binding chain with periodic boundary conditions is given by $E_0 \sum_n |n\rangle\langle n| - t \sum_n (|n\rangle\langle n+1| + |n+1\rangle\langle n|)$. The periodic boundary conditions is manifested by the $+$ in $n+1$ being within $\text{Fin } T.N$ (that is modulo $T.N$).

```
noncomputable def hamiltonian : T.HilbertSpace →l[ℂ] T.HilbertSpace
```

semiformal_result hamiltonian_hermitian[\[source\]](#)

The hamiltonian of the tight binding chain is hermitian.

```
semiformal_result "BUEDT" hamiltonian_hermitian (ψ φ : T.HilbertSpace) :
  ⟨⟨T.hamiltonian ψ, φ⟩⟩_ℂ = ⟨⟨ψ, T.hamiltonian φ⟩⟩_ℂ
```

lemma hamiltonian_apply_localizedState[\[source\]](#)

The Hamiltonian applied to the localized state $|n\rangle$ gives $T.E_0 \cdot |n\rangle - T.t \cdot (|n+1\rangle + |n-1\rangle)$.

```
lemma hamiltonian_apply_localizedState (n : Fin T.N) :
  T.hamiltonian |n⟩ = (T.E_0 : ℂ) • |n⟩ - (T.t : ℂ) • (|n+1⟩ + |n-1⟩)
```

lemma energy_localizedState[\[source\]](#)

The energy of a localized state in the tight binding chain is E_0 . This lemma assumes that there is more than one site in the chain otherwise the result is not true.

```
lemma energy_localizedState (n : Fin T.N) (htn : 1 < T.N) : ⟨⟨|n⟩, T.hamiltonian |n⟩⟩_ℂ = T.E_0
```

def BrillouinZone[\[source\]](#)

The Brillouin zone of the tight binding model is $[-\pi/a, \pi/a)$. This is the set in which wave functions are uniquely defined.

```
def BrillouinZone : Set ℝ
```

def QuantaWaveNumber[\[source\]](#)

*The wavenumbers associated with the energy eigenstates. This corresponds to the set $2\pi / (a N) * (n - \lfloor N/2 \rfloor)$ for $n : \text{Fin } T.N$. It is defined as such so it sits in the Brillouin zone.*

```
def QuantaWaveNumber : Set ℝ
```

lemma quantaWaveNumber_subset_brillouinZone[\[source\]](#)

The quantized wavenumbers form a subset of the BrillouinZone.

```
lemma quantaWaveNumber_subset_brillouinZone : T.QuantaWaveNumber ⊆ T.BrillouinZone
```

lemma quantaWaveNumber_exp_N[\[source\]](#)

```
lemma quantaWaveNumber_exp_N (n : ℕ) (k : T.QuantaWaveNumber) :  
  Complex.exp (Complex.I * k * n * T.N * T.a) = 1
```

lemma quantaWaveNumber_exp_sub_one[\[source\]](#)

```
lemma quantaWaveNumber_exp_sub_one (n : Fin T.N) (k : T.QuantaWaveNumber) :  
  Complex.exp (Complex.I * k * (n - 1).val * T.a) =  
  Complex.exp (Complex.I * k * n * T.a) * Complex.exp (- Complex.I * k * T.a)
```

lemma quantaWaveNumber_exp_add_one[\[source\]](#)

```
lemma quantaWaveNumber_exp_add_one (n : Fin T.N) (k : T.QuantaWaveNumber) :  
  Complex.exp (Complex.I * k * (n + 1).val * T.a) =  
  Complex.exp (Complex.I * k * n * T.a) * Complex.exp (Complex.I * k * T.a)
```

def energyEigenstate[\[source\]](#)

*The energy eigenstates of the tight binding chain They are given by $\sum_n \exp(I * k * n * T.a) \cdot |n\rangle$.*

```
noncomputable def energyEigenstate (k : T.QuantaWaveNumber) : T.HilbertSpace
```

semiformal_result energyEigenstate_orthogonal[\[source\]](#)

The energy eigenstates of the tight binding chain are orthogonal.

```
semiformal_result "BUDDT" energyEigenstate_orthogonal :  
  Pairwise fun k1 k2 => ⟨⟨T.energyEigenstate k1, T.energyEigenstate k2⟩⟩_ℂ = 0
```

def energyEigenvalue[\[source\]](#)

*The energy eigenvalue of the tight binding chain for a k in QuantaWaveNumber is $E_0 - 2 * t * \text{Real.cos } (k * T.a)$.*

```
noncomputable def energyEigenvalue (k : T.QuantaWaveNumber) : ℝ
```

lemma hamiltonian_energyEigenstate[\[source\]](#)

The energy eigenstates satisfy the time-independent Schrodinger equation.

```
lemma hamiltonian_energyEigenstate (k : T.QuantaWaveNumber) :
  T.hamiltonian (T.energyEigenstate k) = T.energyEigenvalue k • T.energyEigenstate k
```

2.6 PhysLean.Electromagnetism.Wave

[PhysLean/Electromagnetism/Wave.lean](#) on GitHub.

theorem waveEquation_electricField_of_freeMaxwellEquations[\[source\]](#)

The electromagnetic wave equation for electric field.

```
theorem waveEquation_electricField_of_freeMaxwellEquations
  (E : ElectricField) (B : MagneticField) (h : OM.FreeMaxwellEquations E B)
  (hE : ContDiff ℝ 2 1 E) (hB : ContDiff ℝ 2 1 B) :
  WaveEquation E t x ((√(μ • ε))⁻¹)
```

theorem waveEquation_magneticField_of_freeMaxwellEquations[\[source\]](#)

The electromagnetic wave equation for magnetic field.

```
theorem waveEquation_magneticField_of_freeMaxwellEquations
  (E : ElectricField) (B : MagneticField) (h : OM.FreeMaxwellEquations E B)
  (hE : ContDiff ℝ 2 1 E) (hB : ContDiff ℝ 2 1 B) :
  WaveEquation B t x ((√(μ • ε))⁻¹)
```

def electricPlaneWave[\[source\]](#)

An electric plane wave travelling in the direction of s with propagation speed c .

```
@[nolint unusedArguments]
noncomputable def electricPlaneWave (E₀ : ℝ → EuclideanSpace ℝ (Fin 3))
  (c : ℝ) (s : Space) (hs : inner ℝ s s = (1:ℝ)) : ElectricField
```

def magneticPlaneWave[\[source\]](#)

A magnetic plane wave travelling in the direction of s with propagation speed c .

```
@[nolint unusedArguments]
noncomputable def magneticPlaneWave (B₀ : ℝ → EuclideanSpace ℝ (Fin 3))
  (c : ℝ) (s : Space) (hs : inner ℝ s s = (1:ℝ)) : MagneticField
```

lemma transverse_upto_time_fun_of_eq_electricPlaneWave[\[source\]](#)

An electric plane wave minus a constant field is transverse for all x .

```
lemma transverse_upto_time_fun_of_eq_electricPlaneWave {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {hs : inner ℝ s s = 1} {E : ElectricField}
  {B : MagneticField} (hEwave : E = electricPlaneWave E₀ c s hs)
  (h' : Differentiable ℝ E₀) (hm : OM.FreeMaxwellEquations E B) :
  ∃ (c : Time → EuclideanSpace ℝ (Fin 3)), ∀ t x, inner ℝ (E t x) s = inner ℝ (c t) s
```

lemma transverse_upto_time_fun_of_eq_magneticPlaneWave[\[source\]](#)

An magnetic plane wave minus a constant field is transverse for all x .

```
lemma transverse_upto_time_fun_of_eq_magneticPlaneWave {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {hs : inner ℝ s s = 1} {E : ElectricField}
  {B : MagneticField} (hBwave : B = magneticPlaneWave B₀ c s hs)
  (h' : Differentiable ℝ B₀) (hm : OM.FreeMaxwellEquations E B) :
  ∃ (c : Time → EuclideanSpace ℝ (Fin 3)), ∀ t x, inner ℝ (B t x) s = inner ℝ (c t) s
```

lemma time_deriv_electricPlaneWave_eq_cross_time_deriv_magneticPlaneWave[\[source\]](#)

The time derivative of a magnetic planewave induces an electric field with time derivative equal to $-s \times_{e3} B'$.

```
lemma time_deriv_electricPlaneWave_eq_cross_time_deriv_magneticPlaneWave
  {t : Time} {x : Space} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {hs : inner ℝ s s = 1} {E : ElectricField} {B : MagneticField}
  (hc : c = (√(μ • ε))⁻¹) (hBwave : B = magneticPlaneWave B₀ c s hs)
  (h' : Differentiable ℝ B₀) (hm : OM.FreeMaxwellEquations E B) :
  ∂ₜ (fun t => E t x) t = - (√(μ • ε))⁻¹ • (s ×ₑ₃ (∂ₜ (fun t => B t x) t))
```

lemma time_deriv_magneticPlaneWave_eq_cross_time_deriv_electricPlaneWave[\[source\]](#)

The time derivative of an electric planewave induces a magnetic field with time derivative equal to $s \times_{e3} E'$.

```
lemma time_deriv_magneticPlaneWave_eq_cross_time_deriv_electricPlaneWave
  {t : Time} {x : Space} {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {s : Space} {hs : inner ℝ s s = 1} {E : ElectricField} {B : MagneticField}
  (hc : c = (√(μ • ε))⁻¹) (hEwave : E = electricPlaneWave E₀ c s hs)
  (h' : Differentiable ℝ E₀) (hm : OM.FreeMaxwellEquations E B) :
  ∂ₜ (fun t => B t x) t = (√(μ • ε)) • (s ×ₑ₃ (∂ₜ (fun t => E t x) t))
```

lemma electricPlaneWave_eq_cross_magneticPlaneWave_upto_space_fun[\[source\]](#)

A magnetic planewave induces an electric field equal to $-s \times_{e3} B$ plus a constant field.

```
lemma electricPlaneWave_eq_cross_magneticPlaneWave_upto_space_fun
  {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {s : Space} {hs : inner ℝ s s = 1}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hBwave : B = magneticPlaneWave B₀ c s hs) (h' : Differentiable ℝ B₀)
  (hm : OM.FreeMaxwellEquations E B) (hE : Differentiable ℝ E) :
  ∃ (c : Space → EuclideanSpace ℝ (Fin 3)), ∀ t x,
  (E t x) = - (√(μ • ε))⁻¹ • (s ×ₑ₃ (B t x)) + c x
```

lemma magneticPlaneWave_eq_cross_electricPlaneWave_upto_space_fun[\[source\]](#)

An electric planewave induces an magnetic field equal to $s \times_{e3} E$ plus a constant field.

```
lemma magneticPlaneWave_eq_cross_electricPlaneWave_upto_space_fun
  {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {s : Space} {hs : inner ℝ s s = (1:ℝ)}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hEwave : E = electricPlaneWave E₀ c s hs) (h' : Differentiable ℝ E₀)
  (hm : OM.FreeMaxwellEquations E B) (hB : Differentiable ℝ B) :
```

$$\exists (c : \text{Space} \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin 3)}), \forall t x,$$

$$(B \ t \ x) = (\sqrt{(\mu \cdot \epsilon)}) \cdot (s \times_{e3} (E \ t \ x)) + c \ x$$
theorem electricField_transverse_upto_const_of_EMwave[\[source\]](#)

The electric field of an EMwave minus a constant field is transverse.

```
theorem electricField_transverse_upto_const_of_EMwave {s : Space} {hs : inner ℝ s s = 1}
  {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hEwave : E = electricPlaneWave E₀ c s hs)
  (hBwave : B = magneticPlaneWave B₀ c s hs)
  (hE' : Differentiable ℝ E₀) (hB' : Differentiable ℝ B₀)
  (hm : OM.FreeMaxwellEquations E B) :
  ∃ (c : EuclideanSpace ℝ (Fin 3)), ∀ t x, inner ℝ (E t x - c) s = 0
```

theorem magneticField_transverse_upto_const_of_EMwave[\[source\]](#)

The magnetic field of an EMwave minus a constant field is transverse.

```
theorem magneticField_transverse_upto_const_of_EMwave {s : Space} {hs : inner ℝ s s = 1}
  {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hEwave : E = electricPlaneWave E₀ c s hs)
  (hBwave : B = magneticPlaneWave B₀ c s hs)
  (hE' : Differentiable ℝ E₀) (hB' : Differentiable ℝ B₀)
  (hm : OM.FreeMaxwellEquations E B) :
  ∃ (c : EuclideanSpace ℝ (Fin 3)), ∀ t x, inner ℝ (B t x - c) s = 0
```

lemma electricField_add_cross_magneticField_eq_const_of_planeWave[\[source\]](#)

E + s ×_{e3} B is constant for an EMwave.

```
lemma electricField_add_cross_magneticField_eq_const_of_planeWave
  {s : Space} {hs : inner ℝ s s = 1}
  {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hEwave : E = electricPlaneWave E₀ c s hs)
  (hBwave : B = magneticPlaneWave B₀ c s hs)
  (hE' : Differentiable ℝ E₀) (hB' : Differentiable ℝ B₀)
  (hm : OM.FreeMaxwellEquations E B) :
  ∃ (Ec : EuclideanSpace ℝ (Fin 3)), ∀ t x,
  (E t x) + (√(μ • ε))⁻¹ • (s ×e3 (B t x)) = Ec
```

lemma magneticField_sub_cross_electricField_eq_const_of_planeWave[\[source\]](#)

B - s ×_{e3} E is constant for an EMwave.

```
lemma magneticField_sub_cross_electricField_eq_const_of_planeWave
  {s : Space} {hs : inner ℝ s s = 1}
  {E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
  {E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
  (hEwave : E = electricPlaneWave E₀ c s hs)
  (hBwave : B = magneticPlaneWave B₀ c s hs)
  (hE' : Differentiable ℝ E₀) (hB' : Differentiable ℝ B₀)
  (hm : OM.FreeMaxwellEquations E B) :
```

```

    ∃ (Ec : EuclideanSpace ℝ (Fin 3)), ∀ t x,
    (B t x) - (√(μ • ε)) • (s ×e3 (E t x)) = Ec

```

lemma forall_fin_three[\[source\]](#)

Pending #25205.

```

lemma _root_.Fin.forall_fin_three {p : Fin 3 → Prop} : (∀ i, p i) ↔ p 0 ∧ p 1 ∧ p 2

```

theorem orthonormal_triad_of_electromagneticplaneWave[\[source\]](#)

Unit vectors in the direction of B , E and s form an orthonormal triad for an EMwave after subtracting the appropriate constant fields.

```

theorem orthonormal_triad_of_electromagneticplaneWave {s : Space} {hs : inner ℝ s s = 1}
{E₀ : ℝ → EuclideanSpace ℝ (Fin 3)} {B₀ : ℝ → EuclideanSpace ℝ (Fin 3)}
{E : ElectricField} {B : MagneticField} (hc : c = (√(μ • ε))⁻¹)
(hEwave : E = electricPlaneWave E₀ c s hs)
(hBwave : B = magneticPlaneWave B₀ c s hs)
(hE' : Differentiable ℝ E₀) (hB' : Differentiable ℝ B₀)
(hm : OM.FreeMaxwellEquations E B) :
∃ (Ep Bp : EuclideanSpace ℝ (Fin 3)), ∀ t x,
E t x - Ep ≠ 0 ∧ B t x - Bp ≠ 0 →
Orthonormal ℝ ![(||E t x - Ep||)⁻¹ • (E t x - Ep), ((||B t x - Bp||)⁻¹) • (B t x - Bp), s]

```

2.7 PhysLean.Mathematics.FDerivCurry

[PhysLean/Mathematics/FDerivCurry.lean](#) on GitHub.

lemma uncurry[\[source\]](#)

```

lemma ContDiff.uncurry (f : X → Y → Z) (x : X) (hf : ContDiff ℤ 2 f) :
ContDiff ℤ 2 (f x)

```

2.8 PhysLean.Mathematics.Geometry.Metric.PseudoRiemannian.Defs

[PhysLean/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean](#) on GitHub.

def negDim[\[source\]](#)

The negative dimension (often called the index or negative index of inertia) of a quadratic form q on a finite-dimensional real vector space.

This value is defined by diagonalizing the quadratic form into an equivalent `QuadraticMap.weightedSumSquare` $\mathbb{R} s$, where $s : \text{Fin } (\text{finrank } \mathbb{R} E) \rightarrow \text{SignType}$ assigns 1, 0, or -1 to each component. The `negDim` is the count of components i for which $s\ i = \text{SignType.neg}$.

By Sylvester's Law of Inertia, this count is an invariant of the quadratic form. Geometrically, `negDim q` represents the dimension of any maximal vector subspace on which q is negative definite. This corresponds to O'Neill's Definition 18 (p. 47) of the index ν of a symmetric bilinear form b on V , which is "the largest integer that is the dimension of a subspace $W \subset V$ on which $b|_W$ is negative definite."

```

noncomputable def negDim {E : Type*} [AddCommGroup E]
[Module ℝ E] [FiniteDimensional ℝ E]

```

```
(q : QuadraticForm ℝ E) : ℕ
```

lemma weightedSumSquares_basis_vector

[\[source\]](#)

For a standard basis vector in a weighted sum of squares, only one term in the sum is nonzero.

```
lemma QuadraticMap.weightedSumSquares_basis_vector {E : Type*} [AddCommGroup E]
[Module ℝ E] {weights : Fin (finrank ℝ E) → ℝ}
{i : Fin (finrank ℝ E)} (v : Fin (finrank ℝ E) → ℝ)
(hv : ∀ j, v j = if j = i then 1 else 0) :
QuadraticMap.weightedSumSquares ℝ weights v = weights i
```

lemma neg_weight_implies_neg_value

[\[source\]](#)

When a quadratic form is equivalent to a weighted sum of squares, negative weights correspond to vectors where the form takes negative values. This is a concrete realization of a 1-dimensional negative definite subspace, contributing to O'Neill's index ν (Definition 18, p. 47).

```
lemma neg_weight_implies_neg_value {E : Type*} [AddCommGroup E] [Module ℝ E]
{q : QuadraticForm ℝ E} {w : Fin (finrank ℝ E) → SignType}
(h_equiv : QuadraticMap.Equivalent q (QuadraticMap.weightedSumSquares ℝ fun i => (w i : ℝ))
)
{i : Fin (finrank ℝ E)} (hi : w i = SignType.neg) :
∃ v : E, v ≠ 0 ∧ q v < 0
```

lemma posDef_no_neg_weights

[\[source\]](#)

A positive definite quadratic form cannot have any negative weights in its diagonal representation. A quadratic form q derived from a bilinear form b is positive definite if $b(v, v) > 0$ for $v \neq 0$ (O'Neill, Definition 17 (1), p. 46). The existence of a negative weight would imply $q(v) < 0$ for some $v \neq 0$, a contradiction.

```
lemma posDef_no_neg_weights {E : Type*} [AddCommGroup E] [Module ℝ E]
{q : QuadraticForm ℝ E} (hq : q.PosDef)
{w : Fin (finrank ℝ E) → SignType}
(h_equiv : QuadraticMap.Equivalent q (QuadraticMap.weightedSumSquares ℝ fun i => (w i : ℝ))
) :
∀ i, w i ≠ SignType.neg
```

theorem rankNeg_eq_zero

[\[source\]](#)

For a positive definite quadratic form, the negative dimension (index) is zero. O'Neill states (p. 47) that " $\nu = 0$ if and only if b is positive semidefinite." Since positive definite implies positive semidefinite (Definitions 17 (1) and (2), p. 46), a positive definite form must have index $\nu = 0$.

```
theorem rankNeg_eq_zero {E : Type*} [AddCommGroup E]
[Module ℝ E] [FiniteDimensional ℝ E] {q : QuadraticForm ℝ E} (hq : q.PosDef) :
q.negDim = 0
```

def pseudoRiemannianMetricValToQuadraticForm

[\[source\]](#)

Constructs a *QuadraticForm* on the tangent space $T_x M$ at a point x from the value of a pseudo-Riemannian metric at that point. (O'Neill, p. 47, "The function $q: V \rightarrow R$ given by $q(v) = b(v, v)$ is the associated quadratic form of b .") The pseudo-Riemannian metric is given by *val*, a family of continuous bilinear forms $g_x: T_x M \times T_x M \rightarrow \mathbb{R}$ for each $x : M$. The quadratic form Q_x at x is defined as $Q_x(v) = g_x(v, v)$. The associated symmetric bilinear form required by *QuadraticForm.exists_companion* is $B_x(v, w) = g_x(v, w) + g_x(w, v)$. Given the symmetry *symm*, this is $2 * g_x(v, w)$.

```
private def pseudoRiemannianMetricValToQuadraticForm
  {E : Type v} [NormedAddCommGroup E] [NormedSpace ℝ E]
  {H : Type w} [TopologicalSpace H]
  {M : Type w} [TopologicalSpace M] [ChartedSpace H M]
  {I : ModelWithCorners ℝ E H}
  (val : ∀ (x : M), TangentSpace I x → L[ℝ] (TangentSpace I x → L[ℝ] ℝ))
  (symm : ∀ (x : M) (v w : TangentSpace I x), (val x v) w = (val x w) v)
  (x : M) : QuadraticForm ℝ (TangentSpace I x) where
```

structure PseudoRiemannianMetric

[\[source\]](#)

A pseudo-Riemannian metric of smoothness class C^n on a manifold M modelled on (E, H) with model I . This structure defines a smoothly varying, non-degenerate, symmetric, continuous bilinear form g_x of constant negative dimension on the tangent space $T_x M$ at each point x . Requires M to be C^{n+1} smooth. This structure formalizes O'Neill's Definition 3.1 (p. 54) of a metric tensor g on M as a "symmetric non-degenerate $(0,2)$ tensor field on M of constant index." Each g_x is a scalar product (O'Neill, Definition 20, p. 47) on $T_x M$.

```
@[ext]
structure PseudoRiemannianMetric
  (E : Type v) (H : Type w) (M : Type w) (n : WithTop ℕ∞)
  [inst_norm_grp_E : NormedAddCommGroup E]
  [inst_norm_sp_E : NormedSpace ℝ E]
  [inst_top_H : TopologicalSpace H]
  [inst_top_M : TopologicalSpace M]
  [inst_chart_M : ChartedSpace H M]
  [inst_chart_E : ChartedSpace H E]
  (I : ModelWithCorners ℝ E H)
  [inst_manifold : IsManifold I (n + 1) M]
  [inst_tangent_findim : ∀ (x : M), FiniteDimensional ℝ (TangentSpace I x)] :
  Type (max u v w) where
  val : ∀ (x : M), TangentSpace I x → L[ℝ] (TangentSpace I x → L[ℝ] ℝ)
  symm : ∀ (x : M) (v w : TangentSpace I x), (val x v) w = (val x w) v
  nondegenerate : ∀ (x : M) (v : TangentSpace I x), (∀ w : TangentSpace I x,
    (val x v) w = 0) → v = 0
  smooth_in_charts' : ∀ (x₀ : M) (v w : E),
    let e := extChartAt I x₀
    ContDiffWithinAt ℝ n
      (fun y => val (e.symm y) (mfderiv I I e.symm y v) (mfderiv I I e.symm y w))
      (e.target) (e x₀)
  negDim_isLocallyConstant :
    IsLocallyConstant (fun x : M =>
      have : FiniteDimensional ℝ (TangentSpace I x) := inferInstance
        (pseudoRiemannianMetricValToQuadraticForm val symm x).negDim)
```

def toBilinForm

[\[source\]](#)

Given a pseudo-Riemannian metric g on manifold M and a point $x : M$, this function constructs a bilinear form on the tangent space at x . For tangent vectors $u, v : T_x M$, the bilinear form is given by: $g_x(u, v) = g(x)(u, v)$

```
def toBilinForm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  LinearMap.BilinForm ℝ (TangentSpace I x) where
```

abbrev toQuadraticForm

[\[source\]](#)

Convert a pseudo-Riemannian metric at a point x to a quadratic form $v \mapsto g_x(v, v)$.

```
abbrev toQuadraticForm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  QuadraticForm ℝ (TangentSpace I x)
```

instance coeFunInst

[\[source\]](#)

Coercion from *PseudoRiemannianMetric* to its function representation.

```
instance coeFunInst : CoeFun (PseudoRiemannianMetric E H M n I)
  (fun _ => ∀ x : M, TangentSpace I x → L[ℝ] (TangentSpace I x → L[ℝ] ℝ)) where
```

lemma toBilinForm_apply

[\[source\]](#)

```
@[simp]
lemma toBilinForm_apply (g : PseudoRiemannianMetric E H M n I) (x : M)
  (v w : TangentSpace I x) :
  toBilinForm g x v w = g.val x v w
```

lemma toQuadraticForm_apply

[\[source\]](#)

```
@[simp]
lemma toQuadraticForm_apply (g : PseudoRiemannianMetric E H M n I) (x : M)
  (v : TangentSpace I x) :
  toQuadraticForm g x v = g.val x v v
```

lemma toBilinForm_isSymm

[\[source\]](#)

```
@[simp]
lemma toBilinForm_isSymm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (toBilinForm g x).IsSymm
```

lemma toBilinForm_nondegenerate

[\[source\]](#)

```
@[simp]
lemma toBilinForm_nondegenerate (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (toBilinForm g x).Nondegenerate
```

def inner

[\[source\]](#)

The inner product (or scalar product) on the tangent space at point x induced by the pseudo-Riemannian metric g . This is $g_x(v, w)$.

```
def inner (g : PseudoRiemannianMetric E H M n I) (x : M) (v w : TangentSpace I x) : ℝ
```

lemma inner_apply[\[source\]](#)

```
@[simp]
lemma inner_apply (g : PseudoRiemannianMetric E H M n I) (x : M) (v w : TangentSpace I x) :
  inner g x v w = g.val x v w
```

def flat[\[source\]](#)

The "musical" isomorphism (index lowering) $v \mapsto g_x(v, -)$. The non-degeneracy of g_x (O'Neill, Def 17 (3), p. 46) means its matrix representation is invertible (O'Neill, Lemma 19, p. 47), and that this map is an isomorphism from $T_x M$ to its dual.

```
def flat (g : PseudoRiemannianMetric E H M n I) (x : M) :
  TangentSpace I x →ℓ[ℝ] (TangentSpace I x →L[ℝ] ℝ)
```

lemma flat_apply[\[source\]](#)

```
@[simp]
lemma flat_apply (g : PseudoRiemannianMetric E H M n I) (x : M) (v w : TangentSpace I x) :
  (flat g x v) w = g.val x v w
```

def flatL[\[source\]](#)

The musical isomorphism as a continuous linear map.

```
def flatL (g : PseudoRiemannianMetric E H M n I) (x : M) :
  TangentSpace I x →L[ℝ] (TangentSpace I x →L[ℝ] ℝ) where
```

lemma flatL_apply[\[source\]](#)

```
@[simp]
lemma flatL_apply (g : PseudoRiemannianMetric E H M n I) (x : M) (v w : TangentSpace I x) :
  (flatL g x v) w = g.val x v w
```

lemma flat_inj[\[source\]](#)

```
@[simp]
lemma flat_inj (g : PseudoRiemannianMetric E H M n I) (x : M) :
  Function.Injective (flat g x)
```

lemma flatL_inj[\[source\]](#)

```
@[simp]
lemma flatL_inj (g : PseudoRiemannianMetric E H M n I) (x : M) :
  Function.Injective (flatL g x)
```

lemma flatL_surj[\[source\]](#)

```
@[simp]
lemma flatL_surj
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  Function.Surjective (g.flatL x)
```

def flatEquiv[\[source\]](#)

The “musical” isomorphism (index lowering) from T^*M to its dual, as a continuous linear equivalence. This equivalence is a direct result of g_x being a non-degenerate bilinear form (O’Neill, Def 17(3), p. 46; Lemma 19, p. 47).

```
def flatEquiv
  (g : PseudoRiemannianMetric E H M n I)
  (x : M) :
  TangentSpace I x  $\approx$ L[ $\mathbb{R}$ ] (TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ]  $\mathbb{R}$ )
```

lemma coe_flatEquiv[\[source\]](#)

```
lemma coe_flatEquiv
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (g.flatEquiv x : TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ] (TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ]  $\mathbb{R}$ )) = g.flatL x
```

lemma flatEquiv_apply[\[source\]](#)

```
@[simp]
lemma flatEquiv_apply
  (g : PseudoRiemannianMetric E H M n I) (x : M) (v w : TangentSpace I x) :
  (g.flatEquiv x v) w = g.val x v w
```

def sharpEquiv[\[source\]](#)

The “musical” isomorphism (index raising) from the dual of T^*M to T^*M . This is the inverse of `flatEquiv g x`, and its existence as an isomorphism is guaranteed by the non-degeneracy of g_x (O’Neill, Lemma 19, p. 47).

```
def sharpEquiv
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ]  $\mathbb{R}$ )  $\approx$ L[ $\mathbb{R}$ ] TangentSpace I x
```

def sharpL[\[source\]](#)

The index raising map *sharp* as a continuous linear map.

```
def sharpL
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ]  $\mathbb{R}$ )  $\rightarrow$ L[ $\mathbb{R}$ ] TangentSpace I x
```

lemma sharpL_eq_toContinuousLinearMap[\[source\]](#)

```
lemma sharpL_eq_toContinuousLinearMap
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  g.sharpL x = (g.sharpEquiv x).toContinuousLinearMap
```

lemma coe_sharpEquiv[\[source\]](#)

```
lemma coe_sharpEquiv
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (g.sharpEquiv x : (TangentSpace I x  $\rightarrow$ L[ $\mathbb{R}$ ]  $\mathbb{R}$ )  $\rightarrow$ L[ $\mathbb{R}$ ] TangentSpace I x) = g.sharpL x
```

def sharp[\[source\]](#)

The index raising map sharp as a linear map.

```
noncomputable def sharp
  (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (TangentSpace I x →L[ℝ] ℝ) →ℓ[ℝ] TangentSpace I x
```

lemma sharpL_apply_flatL[\[source\]](#)

```
@[simp]
lemma sharpL_apply_flatL
  (g : PseudoRiemannianMetric E H M n I) (x : M) (v : TangentSpace I x) :
  g.sharpL x (g.flatL x v) = v
```

lemma flatL_apply_sharpL[\[source\]](#)

```
@[simp]
lemma flatL_apply_sharpL
  (g : PseudoRiemannianMetric E H M n I) (x : M) (ω : TangentSpace I x →L[ℝ] ℝ) :
  g.flatL x (g.sharpL x ω) = ω
```

lemma flat_sharp_apply[\[source\]](#)

Applying sharp then flat recovers the original covector.

```
@[simp]
lemma flat_sharp_apply
  (g : PseudoRiemannianMetric E H M n I) (x : M) (ω : TangentSpace I x →L[ℝ] ℝ) :
  g.flat x (g.sharp x ω) = ω
```

lemma sharp_flat_apply[\[source\]](#)

```
@[simp]
lemma sharp_flat_apply
  (g : PseudoRiemannianMetric E H M n I) (x : M) (v : TangentSpace I x) :
  g.sharp x (g.flat x v) = v
```

lemma apply_sharp_sharp[\[source\]](#)

The metric evaluated at sharp ω_1 and sharp ω_2 .

```
@[simp]
lemma apply_sharp_sharp
  (g : PseudoRiemannianMetric E H M n I) (x : M) (ω1 ω2 : TangentSpace I x →L[ℝ] ℝ) :
  g.val x (g.sharpL x ω1) (g.sharpL x ω2) = ω1 (g.sharpL x ω2)
```

lemma apply_vec_sharp[\[source\]](#)

The metric evaluated at v and sharp ω.

```
lemma apply_vec_sharp
  (g : PseudoRiemannianMetric E H M n I) (x : M) (v : TangentSpace I x)
  (ω : TangentSpace I x →L[ℝ] ℝ) :
  g.val x v (g.sharpL x ω) = ω v
```

def cotangentMetricVal[\[source\]](#)

The value of the induced metric on the cotangent space at point x .

```
noncomputable def cotangentMetricVal (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω1 ω2 : TangentSpace I x →L[ℝ] ℝ) : ℝ
```

lemma cotangentMetricVal_eq_apply_sharp[\[source\]](#)

```
@[simp]
lemma cotangentMetricVal_eq_apply_sharp (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω1 ω2 : TangentSpace I x →L[ℝ] ℝ) :
  cotangentMetricVal g x ω1 ω2 = ω1 (g.sharpL x ω2)
```

lemma cotangentMetricVal_symm[\[source\]](#)

```
lemma cotangentMetricVal_symm (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω1 ω2 : TangentSpace I x →L[ℝ] ℝ) :
  cotangentMetricVal g x ω1 ω2 = cotangentMetricVal g x ω2 ω1
```

def cotangentToBilinForm[\[source\]](#)

The induced metric on the cotangent space at point x as a bilinear form. For covectors ω_1 and ω_2 , this gives $g(\omega_1^\sharp, \omega_2^\sharp)$, where $\omega^\sharp$ is the "sharp" musical isomorphism raising indices.

```
noncomputable def cotangentToBilinForm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  LinearMap.BilinForm ℝ (TangentSpace I x →L[ℝ] ℝ) where
```

def cotangentToQuadraticForm[\[source\]](#)

*The cometric on the cotangent space T_x^*M at x , expressed as a quadratic form. It is induced by the pseudo-Riemannian metric on the tangent space T_xM .*

```
noncomputable def cotangentToQuadraticForm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  QuadraticForm ℝ (TangentSpace I x →L[ℝ] ℝ) where
```

lemma cotangentToBilinForm_apply[\[source\]](#)

```
@[simp]
lemma cotangentToBilinForm_apply (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω1 ω2 : TangentSpace I x →L[ℝ] ℝ) :
  cotangentToBilinForm g x ω1 ω2 = cotangentMetricVal g x ω1 ω2
```

lemma cotangentToQuadraticForm_apply[\[source\]](#)

```
@[simp]
lemma cotangentToQuadraticForm_apply (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω : TangentSpace I x →L[ℝ] ℝ) :
  cotangentToQuadraticForm g x ω = cotangentMetricVal g x ω ω
```

lemma cotangentToBilinForm_isSymm[\[source\]](#)

```
@[simp]
lemma cotangentToBilinForm_isSymm (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (cotangentToBilinForm g x).IsSymm
```

lemma cotangentMetricVal_nondegenerate[\[source\]](#)

The cotangent metric is non-degenerate: if $\text{cotangentMetricVal } g \ x \ \omega \ v = 0$ for all v , then $\omega = 0$.

```
lemma cotangentMetricVal_nondegenerate (g : PseudoRiemannianMetric E H M n I) (x : M)
  (ω : TangentSpace I x →L[ℝ] ℝ) (h : ∀ v : TangentSpace I x →L[ℝ] ℝ,
    cotangentMetricVal g x ω v = 0) :
  ω = 0
```

lemma cotangentToBilinForm_nondegenerate[\[source\]](#)

```
@[simp]
lemma cotangentToBilinForm_nondegenerate (g : PseudoRiemannianMetric E H M n I) (x : M) :
  (cotangentToBilinForm g x).Nondegenerate
```

2.9 PhysLean.Mathematics.Geometry.Metric.Riemannian.Defs

[PhysLean/Mathematics/Geometry/Metric/Riemannian/Defs.lean](#) on GitHub.

structure RiemannianMetric[\[source\]](#)

A RiemannianMetric on a manifold M modeled on H with corners I (over the model space E , typically $\mathbb{R}^m$) is a family of inner products on the tangent spaces $\text{TangentSpace } I \ x$ for each $x : M$. This family is required to vary smoothly with x , specifically with smoothness C^n .

This structure extends `PseudoRiemannianMetric`, inheriting the core properties of a pseudo-Riemannian metric, such as being a symmetric, non-degenerate, C^n -smooth tensor field of type $(0,2)$. The key distinguishing feature of a Riemannian metric is its positive-definiteness.

The `pos_def'` field ensures that for any point x on the manifold and any non-zero tangent vector v at x , the inner product $g_x(v, v)$ (denoted `val x v v`) is strictly positive. This condition makes each `val x` (the metric at point x) a positive-definite symmetric bilinear form, effectively an inner product, on the tangent space $\text{TangentSpace } I \ x$.

Parameters: - `I`: The `ModelWithCorners` for the manifold M . This defines the model space E (e.g., $\mathbb{R}^d$) and the model space for the boundary H . - `n`: The smoothness class of the metric, an $\mathbb{N}$ value. The metric tensor components are C^n functions. - `M`: The type of the manifold. - `[TopologicalSpace M]`: Assumes M has a topological structure. - `[ChartedSpace H M]`: Assumes M is equipped with an atlas of charts to H . - `[IsManifold I (n + 1) M]`: Assumes M is a manifold of smoothness $C^{(n+1)}$. The manifold needs to be slightly smoother than the metric itself for certain constructions. - `[inst_tangent_findim : ∀ (x : M), FiniteDimensional ℝ (TangentSpace I x)]`: Ensures that each tangent space is a finite-dimensional real vector space.

Fields: - `toPseudoRiemannianMetric`: The underlying pseudo-Riemannian metric. This provides the smooth family of symmetric bilinear forms `val : M → SymBilinForm`

$\mathbb{R}$ (TangentSpace I ·). - *pos_def'*: The positive-definiteness condition: $\forall x \ v, \ v \neq 0 \rightarrow \text{val } x \ v \ v > 0$. This asserts that for any point x and any non-zero tangent vector v at x , the metric evaluated on (v, v) is strictly positive.

```
@[ext]
structure RiemannianMetric
  (I : ModelWithCorners  $\mathbb{R}$  E H) (n :  $\mathbb{N}_\infty$ ) (M : Type w)
  [TopologicalSpace M] [ChartedSpace H M] [IsManifold I (n + 1) M]
  [inst_tangent_findim :  $\forall (x : M), \text{FiniteDimensional } \mathbb{R} \text{ (TangentSpace I x)}$ ]
  extends PseudoRiemannianMetric E H M n I where
  pos_def' :  $\forall x \ v, \ v \neq 0 \rightarrow \text{val } x \ v \ v > 0$ 
```

instance Coe (RiemannianMetric I n M) (PseudoRiemannianMetric E H M (n) I) [\[source\]](#)

Coercion from RiemannianMetric to its underlying PseudoRiemannianMetric.

```
instance : Coe (RiemannianMetric I n M) (PseudoRiemannianMetric E H M (n) I) where
```

lemma pos_def [\[source\]](#)

```
@[simp]
lemma pos_def (g : RiemannianMetric I n M) (x : M) (v : TangentSpace I x)
  (hv : v  $\neq$  0) :
  (g.toPseudoRiemannianMetric.val x v) v > 0
```

lemma toQuadraticForm_posDef [\[source\]](#)

The quadratic form associated with a Riemannian metric is positive definite.

```
@[simp]
lemma toQuadraticForm_posDef (g : RiemannianMetric I n M) (x : M) :
  (g.toQuadraticForm x).PosDef
```

lemma riemannian_metric_negDim_zero [\[source\]](#)

```
lemma riemannian_metric_negDim_zero (g : RiemannianMetric I n M) (x : M) :
  (g.toQuadraticForm x).negDim = 0
```

def tangentInnerCore [\[source\]](#)

The InnerProductSpace.Core structure for $T_x M$ induced by a Riemannian metric g . This provides the properties of an inner product: symmetry, non-negativity, linearity, and definiteness. Each g_x is an inner product on $T_x M$ (O'Neill, p. 55).

```
noncomputable def tangentInnerCore (g : RiemannianMetric I n M) (x : M) :
  InnerProductSpace.Core  $\mathbb{R}$  (TangentSpace I x) where
```

def metricNormedAddCommGroup [\[source\]](#)

Creates a NormedAddCommGroup structure on $T_x M$ from a Riemannian metric g .

```
noncomputable def TangentSpace.metricNormedAddCommGroup (g : RiemannianMetric I n M) (x : M) :
  NormedAddCommGroup (TangentSpace I x)
```

def metricInnerProductSpace'[\[source\]](#)

Creates an InnerProductSpace structure on $T_x M$ from a Riemannian metric g . Alternative implementation using `letI`.

```
noncomputable def TangentSpace.metricInnerProductSpace' (g : RiemannianMetric I n M) (x : M) :
  letI
```

def metricInnerProductSpace[\[source\]](#)

Creates an InnerProductSpace structure on $T_x M$ from a Riemannian metric g .

```
noncomputable def TangentSpace.metricInnerProductSpace (g : RiemannianMetric I n M) (x : M) :
  let _
```

def norm[\[source\]](#)

The norm on a tangent space induced by a Riemannian metric, defined as the square root of the inner product of a vector with itself.

```
noncomputable def norm (g : RiemannianMetric I n M) (x : M) (v : TangentSpace I x) : ℝ
```

def norm'[\[source\]](#)

Helper function to compute the norm on a tangent space from a Riemannian metric, using the underlying NormedAddCommGroup structure.

```
noncomputable def norm' (g : RiemannianMetric I n M) (x : M) (v : TangentSpace I x) : ℝ
```

lemma norm_eq_norm_of_metricNormedAddCommGroup[\[source\]](#)

```
lemma norm_eq_norm_of_metricNormedAddCommGroup (g : RiemannianMetric I n M) (x : M)
  (v : TangentSpace I x) : norm g x v = @Norm.norm (TangentSpace I x)
  (@NormedAddCommGroup.toNorm _ (TangentSpace.metricNormedAddCommGroup g x)) v
```

def curveLength[\[source\]](#)

Calculates the length of a curve γ between parameters t_0 and t_1 using the Riemannian metric g . This is defined as the integral of the norm of the tangent vector along the curve.

```
def curveLength (g : RiemannianMetric I n M) (γ : ℝ → M) (t₀ t₁ : ℝ)
  {IDE : ModelWithCorners ℝ ℝ ℝ}[ChartedSpace ℝ ℝ] : ℝ
```

2.10 PhysLean.QuantumMechanics.FiniteTarget.HilbertSpace

[PhysLean/QuantumMechanics/FiniteTarget/HilbertSpace.lean](#) on GitHub.

def FiniteHilbertSpace[\[source\]](#)

The finite dimensional Hilbert space of dimension n .

```
def FiniteHilbertSpace (n : ℕ) : Type
```


instance AddCommGroup (FiniteHilbertSpace n) [\[source\]](#)

```
instance {n : ℕ} : AddCommGroup (FiniteHilbertSpace n)
```

instance Module ℂ (FiniteHilbertSpace n) [\[source\]](#)

```
noncomputable instance {n : ℕ} : Module ℂ (FiniteHilbertSpace n)
```

instance NormedAddCommGroup (FiniteHilbertSpace n) [\[source\]](#)

```
noncomputable instance {n : ℕ} : NormedAddCommGroup (FiniteHilbertSpace n)
```

instance InnerProductSpace ℂ (FiniteHilbertSpace n) [\[source\]](#)

```
noncomputable instance {n : ℕ} : InnerProductSpace ℂ (FiniteHilbertSpace n)
```

instance CompleteSpace (FiniteHilbertSpace n) [\[source\]](#)

```
noncomputable instance {n : ℕ} : CompleteSpace (FiniteHilbertSpace n)
```

2.11 PhysLean.Relativity.PauliMatrices.CliffordAlgebra

[PhysLean/Relativity/PauliMatrices/CliffordAlgebra.lean](#) on GitHub.

def form [\[source\]](#)

The euclidean norm as a quadratic form.

```
@[simps!]
protected noncomputable def form : QuadraticForm ℝ (Fin 3 → ℝ)
```

def ofCliffordAlgebra [\[source\]](#)

The injection from the Clifford algebra over PauliMatrix.form into the algebra generated by the σ matrices.

```
def ofCliffordAlgebra :
  CliffordAlgebra PauliMatrix.form →ₐ[ℝ] Algebra.adjoin ℝ {σ1, σ2, σ3}
```

lemma ofCliffordAlgebra_ι_single [\[source\]](#)

The generators of the Clifford algebra correspond to the elements σ .

```
@[simp]
lemma ofCliffordAlgebra_ι_single (i : Fin 3) (r : ℝ) :
  ofCliffordAlgebra (CliffordAlgebra.ι _ (Pi.single i r)) =
    r • (![σ1, σ2, σ3] i, Algebra.subset_adjoin <| by fin_cases i <=> simp)
```

2.12 PhysLean.Relativity.SpaceTime.CliffordAlgebra

[PhysLean/Relativity/SpaceTime/CliffordAlgebra.lean](#) on GitHub.

theorem one_fin_four[\[source\]](#)

```
theorem _root_.Matrix.one_fin_four {α} [Zero α] [One α] :
  (1 : Matrix (Fin 4) (Fin 4) α) = !![1, 0, 0, 0; 0, 1, 0, 0; 0, 0, 1, 0; 0, 0, 0, 1]
```

def diracForm[\[source\]](#)

The quadratic form of the clifford algebra corresponding to the γ matrices.

```
@[simps!]
def diracForm : QuadraticForm ℝ (Fin 4 → ℝ)
```

def ofCliffordAlgebra[\[source\]](#)

*The injection from the clifford algebra over **diracForm** into the algebra generated by the γ matrices.*

```
def ofCliffordAlgebra : CliffordAlgebra diracForm →a [ℝ] diracAlgebra
```

lemma ofCliffordAlgebra_ι_single[\[source\]](#)

The generators of the clifford algebra correspond to the elements γ .

```
@[simp]
lemma ofCliffordAlgebra_ι_single (i : Fin 4) (r : ℝ) :
  ofCliffordAlgebra (CliffordAlgebra.ι _ (Pi.single i r)) = r • ⟨γ i, γ_in_diracAlgebra _⟩
```

2.13 PhysLean.Relativity.Tensors.Color.Basic

[PhysLean/Relativity/Tensors/Color/Basic.lean](#) on GitHub.

abbrev OverColor.hom[\[source\]](#)

*The underlying morphism in the category of Types of a object f in **OverColor** C .*

```
abbrev hom (f : OverColor C)
```

abbrev left[\[source\]](#)

*The underlying object in the category of Types of a object f in **OverColor** C .*

```
abbrev left (f : OverColor C)
```

abbrev CoreHom.hom[\[source\]](#)

*The underlying morphism in the category of Types of a morphism m in **OverColor** C .*

```
abbrev _root_.CategoryTheory.CoreHom.hom (m : f → g)
```

abbrev inv[\[source\]](#)

*The underlying inverse-morphism in the category of Types of a morphism m in **OverColor** C .*

```
abbrev _root_.CategoryTheory.CoreHom.inv (m : f → g)
```

lemma hom_inv_id[\[source\]](#)

```
lemma _root_.CategoryTheory.CoreHom.hom_inv_id (m : f → g) : m.iso.hom >> m.iso.inv =  $\mathbb{K}$  f.of
```

lemma inv_hom_id[\[source\]](#)

```
lemma _root_.CategoryTheory.CoreHom.inv_hom_id (m : f → g) : m.iso.inv >> m.iso.hom =  $\mathbb{K}$  g.of
```

abbrev symm[\[source\]](#)

The inverse of a morphism in OverColor C.

```
abbrev _root_.CategoryTheory.CoreHom.symm (m : f → g) : g → f
```

2.14 PhysLean.Relativity.Tensors.Color.Lift

[PhysLean/Relativity/Tensors/Color/Lift.lean](#) on GitHub.

def toRep[\[source\]](#)

Given an object $f : \text{OverColor } C$ and a function $F : \text{Discrete } C \rightsquigarrow \text{Rep } k \ G$, the object in $\text{Rep } k \ G$ obtained by taking the tensor product of all colors in f .

```
def toRep (f : OverColor C) : Rep k G
```

lemma toRep_ρ[\[source\]](#)

```
lemma toRep_ρ (f : OverColor C) (M : G) : (toRep F f).ρ M =  
  PiTensorProduct.map (fun x => (F.obj (Discrete.mk (f.hom x))).ρ M)
```

lemma toRep_ρ_tprod[\[source\]](#)

```
lemma toRep_ρ_tprod (f : OverColor C) (M : G) (x : (i : f.left) → F.obj (Discrete.mk (f.hom i))  
  ) :  
  (toRep F f).ρ M (PiTensorProduct.tprod k x) =  
  PiTensorProduct.tprod k fun i => (F.obj (Discrete.mk (f.hom i))).ρ M (x i)
```

lemma toRep_ρ_empty[\[source\]](#)

```
lemma toRep_ρ_empty (g : G) : (toRep F ( $\mathbb{K}$ _ (OverColor C))).ρ g = LinearMap.id
```

lemma toRep_ρ_from_fin0[\[source\]](#)

```
lemma toRep_ρ_from_fin0 (c : Fin 0 → C) (g : G) :  
  (toRep F (OverColor.mk c)).ρ g = LinearMap.id
```

lemma toRep_V_carrier[\[source\]](#)

```
lemma toRep_V_carrier (f : OverColor C) :  
  (toRep F f).V =  $\bigotimes [k] (i : f.left), F.obj (Discrete.mk (f.hom i))$ 
```

def linearIsoOfEq[\[source\]](#)

For a function $F : \text{Discrete } C \rightsquigarrow \text{Rep } k \ G$, the linear equivalence $(F.\text{obj } c1).V \approx_l[k] (F.\text{obj } c2).V$ induced by an equality of $c1$ and $c2$.

```
def linearIsoOfEq {c1 c2 : Discrete C} (h : c1.as = c2.as) :
  (F.obj c1).V ≈l[k] (F.obj c2).V
```

lemma linearIsoOfEq_comm_p[\[source\]](#)

```
lemma linearIsoOfEq_comm_p {c1 c2 : Discrete C} (h : c1.as = c2.as) (M : G)
  (x : F.obj c1) : linearIsoOfEq F h ((F.obj c1).p M x) =
  (F.obj c2).p M (linearIsoOfEq F h x)
```

def linearIsoOfHom[\[source\]](#)

Given a morphism in $\text{OverColor } C$, $m : f \longrightarrow g$ and a functor $F : \text{Discrete } C \rightsquigarrow \text{Rep } k \ G$, the linear equivalence $(\text{toRep } F f).V \approx_l[k] (\text{toRep } F g).V$ formed by reindexing.

```
def linearIsoOfHom {f g : OverColor C} (m : f → g) : (toRep F f).V ≈l[k] (toRep F g).V
```

lemma linearIsoOfHom_tprod[\[source\]](#)

```
lemma linearIsoOfHom_tprod {f g : OverColor C} (m : f → g)
  (x : (i : f.left) → (F.obj (Discrete.mk (f.hom i)))) :
  linearIsoOfHom F m (PiTensorProduct.tprod k x) =
  PiTensorProduct.tprod k (fun i => (linearIsoOfEq F (Hom.toEquiv_symm_apply m i))
  (x ((OverColor.Hom.toEquiv m).symm i)))
```

def homToRepHom[\[source\]](#)

Given a morphism in $\text{OverColor } C$, $m : f \longrightarrow g$ and a functor $F : \text{Discrete } C \rightsquigarrow \text{Rep } k \ G$, the morphism $(\text{toRep } F f) \longrightarrow (\text{toRep } F g)$ formed by reindexing.

```
def homToRepHom {f g : OverColor C} (m : f → g) : toRep F f → toRep F g where
```

lemma homToRepHom_tprod[\[source\]](#)

```
lemma homToRepHom_tprod {X Y : OverColor C} (p : (i : X.left) → F.obj (Discrete.mk (X.hom i)))
  (f : X → Y) : (homToRepHom F f).hom (PiTensorProduct.tprod k p) =
  PiTensorProduct.tprod k fun i => linearIsoOfEq F
  (OverColor.Hom.toEquiv_comp_inv_apply f i) (p ((OverColor.Hom.toEquiv f).symm i))
```

lemma homToRepHom_id[\[source\]](#)

```
lemma homToRepHom_id (X : OverColor C) : homToRepHom F (≡ X) = ≡ _
```

lemma homToRepHom_comp[\[source\]](#)

```
lemma homToRepHom_comp {X Y Z : OverColor C} (f : X → Y) (g : Y → Z) :
  homToRepHom F (f >> g) = homToRepHom F f >> homToRepHom F g
```

def toRepFunc[\[source\]](#)

The Functor (OverColor C) (Rep k G) from a functor Discrete C $\rightsquigarrow$ Rep k G.

```
def toRepFunc : Functor (OverColor C) (Rep k G) where
```

def toRepUnitIso[\[source\]](#)

The unit isomorphism between $\mathbb{K}_-$ (Rep k G) and toRep F ($\mathbb{K}_-$ (OverColor C)) generated using PiTensorProduct.isEmptyEquiv.

```
def toRepUnitIso :  $\mathbb{K}_-$  (Rep k G)  $\cong$  toRep F ( $\mathbb{K}_-$  (OverColor C))
```

instance toRepFunc_laxBraidedFunctor[\[source\]](#)

The lift of a functor is lax braided.

```
instance toRepFunc_laxBraidedFunctor : Functor.LaxBraided (toRepFunc F) where
```

instance toRepFunc_monoidalFunctor[\[source\]](#)

The lift of a functor is monoidal.

```
instance toRepFunc_monoidalFunctor : Functor.Monoidal (toRepFunc F)
```

instance toRepFunc_braidedFunctor[\[source\]](#)

The lift of a functor is braided.

```
instance toRepFunc_braidedFunctor : Functor.Braided (toRepFunc F)
```

def repNatTransOfColorApp[\[source\]](#)

Given two functors $F F' : \text{Discrete } C \rightsquigarrow \text{Rep } k G$ and a natural transformation $\eta : F \longrightarrow F'$, and an $X : \text{OverColor } C$, the $(\text{toRepFunc } F).obj X \longrightarrow (\text{toRepFunc } F').obj X$ in Rep k G constructed by the tensor product of the morphisms in η with corresponding color.

```
def repNatTransOfColorApp (X : OverColor C) : (toRepFunc F).obj X  $\longrightarrow$  (toRepFunc F').obj X
  where
```

lemma repNatTransOfColorApp_tprod[\[source\]](#)

```
lemma repNatTransOfColorApp_tprod (X : OverColor C)
  (p : (i : X.left)  $\rightarrow$  F.obj (Discrete.mk (X.hom i))) :
  (repNatTransOfColorApp  $\eta$  X).hom (PiTensorProduct.tprod k p) =
  PiTensorProduct.tprod k fun i => ( $\eta$ .app (Discrete.mk (X.hom i))).hom (p i)
```

lemma repNatTransOfColorApp_naturality[\[source\]](#)

```
lemma repNatTransOfColorApp_naturality {X Y : OverColor C} (f : X  $\longrightarrow$  Y) :
  (toRepFunc F).map f  $\gg$  repNatTransOfColorApp  $\eta$  Y =
  repNatTransOfColorApp  $\eta$  X  $\gg$  (toRepFunc F').map f
```

def repNatTransOfColor[\[source\]](#)

Given a natural transformation between $F \text{ } F' : \text{Discrete } C \rightsquigarrow \text{Rep } k \text{ } G$ the monoidal natural transformation between $\text{toRepFunc } F$ and $\text{toRepFunc } F'$.

```
def repNatTransOfColor : (toRepFunc F) → (toRepFunc F') where
```

lemma repNatTransOfColorApp_unit[\[source\]](#)

```
lemma repNatTransOfColorApp_unit : Functor.LaxMonoidal.ε (toRepFunc F) >>
  repNatTransOfColorApp η (⊗_ (OverColor C)) = Functor.LaxMonoidal.ε (toRepFunc F')
```

lemma repNatTransOfColorApp_tensor[\[source\]](#)

```
lemma repNatTransOfColorApp_tensor (X Y : OverColor C) :
  (Functor.LaxMonoidal.μ (toRepFunc F)) X Y >> repNatTransOfColorApp η (X ⊗ Y) =
  (repNatTransOfColorApp η X ⊗ repNatTransOfColorApp η Y) >>
  (Functor.LaxMonoidal.μ (toRepFunc F')) X Y
```

instance repNatTransOfColor_isMonoidal[\[source\]](#)

The lift of a natural transformation is monoidal.

```
instance repNatTransOfColor_isMonoidal : NatTrans.IsMonoidal (repNatTransOfColor η) where
```

2.15 PhysLean.StringTheory.FTheory.SU5U1.AnomalyCancellation.Basic

[PhysLean/StringTheory/FTheory/SU5U1/AnomalyCancellation/Basic.lean](#) on GitHub.

def fiveAnomalyCoefficient[\[source\]](#)

The anomaly coefficients associated with matter in the five-bar representation.

```
def fiveAnomalyCoefficient : ℤ × ℤ
```

def tenAnomalyCoefficient[\[source\]](#)

The anomaly coefficients associated with matter in the ten-dim representation.

```
def tenAnomalyCoefficient : ℤ × ℤ
```

lemma anomalyCoefficient_sum_of_gaugeAnomalyU1YU1U1_gaugeAnomalyU1YU1U1[\[source\]](#)

```
lemma anomalyCoefficient_sum_of_gaugeAnomalyU1YU1U1_gaugeAnomalyU1YU1U1
  (acc1 : ℳ.GaugeAnomalyU1MSSM) (acc2 : ℳ.GaugeAnomalyU1YU1U1) :
  ℳ.fiveAnomalyCoefficient + ℳ.tenAnomalyCoefficient
  - (ℳ.qHu, ℳ.qHu * ℳ.qHu) + (ℳ.qHd, ℳ.qHd * ℳ.qHd) = (0, 0)
```

2.16 PhysLean.StringTheory.FTheory.SU5U1.Charges.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Charges/Basic.lean](#) on GitHub.

def Charges[\[source\]](#)

The type such that an element corresponds to the collection of charges associated with the matter content of the theory. The order of charges is implicitly taken to be qHd , qHu , $Q5$, $Q10$

```
def Charges : Type
```

instance DecidableEq Charges[\[source\]](#)

```
instance : DecidableEq Charges
```

instance Repr Charges[\[source\]](#)

```
unsafe instance : Repr Charges where
```

instance hasSubset[\[source\]](#)

```
instance hasSubset : HasSubset Charges where
```

instance hasSSubset[\[source\]](#)

```
instance hasSSubset : HasSSubset Charges where
```

instance subsetDecidable[\[source\]](#)

```
instance subsetDecidable (x y : Charges) : Decidable (x ⊆ y)
```

lemma subset_refl[\[source\]](#)

```
@[simp, refl]
lemma subset_refl (x : Charges) : x ⊆ x
```

lemma subset_trans[\[source\]](#)

```
lemma subset_trans {x y z : Charges} (hxy : x ⊆ y) (hyz : y ⊆ z) : x ⊆ z
```

lemma subset_antisymm[\[source\]](#)

```
lemma subset_antisymm {x y : Charges} (hxy : x ⊆ y) (hyx : y ⊆ x) : x = y
```

instance emptyInst[\[source\]](#)

```
instance emptyInst : EmptyCollection Charges where
```

lemma empty_subset[\[source\]](#)

```
@[simp]
lemma empty_subset (x : Charges) : ∅ ⊆ x
```

lemma subset_of_empty_iff_empty[\[source\]](#)

```
@[simp]
lemma subset_of_empty_iff_empty {x : Charges} :
  x  $\subseteq$   $\emptyset$   $\leftrightarrow$  x =  $\emptyset$ 
```

def powerset[\[source\]](#)

The powerset of a charge . Given a charge $x : \text{Charges}$ it's powerset is the finite set of all Charges which are subsets of x .

```
def powerset (x : Charges) : Finset Charges
```

lemma mem_powerset_iff_subset[\[source\]](#)

```
@[simp]
lemma mem_powerset_iff_subset {x y : Charges} :
  x  $\in$  powerset y  $\leftrightarrow$  x  $\subseteq$  y
```

lemma self_mem_powerset[\[source\]](#)

```
lemma self_mem_powerset (x : Charges) :
  x  $\in$  powerset x
```

lemma empty_mem_powerset[\[source\]](#)

```
lemma empty_mem_powerset (x : Charges) :
   $\emptyset$   $\in$  powerset x
```

lemma powerset_of_empty[\[source\]](#)

```
@[simp]
lemma powerset_of_empty :
  powerset  $\emptyset$  = { $\emptyset$ }
```

lemma powerset_subset_iff_subset[\[source\]](#)

```
lemma powerset_subset_iff_subset {x y : Charges} :
  powerset x  $\subseteq$  powerset y  $\leftrightarrow$  x  $\subseteq$  y
```

def fromChargeProfile[\[source\]](#)

The collection of charges associated with a charge profile.

```
def fromChargeProfile : (T : PotentialTerm)  $\rightarrow$  T.ChargeProfile  $\rightarrow$  Charges
|  $\mu$ , (qHd, qHu) => (qHd, qHu, {}, {})
|  $\beta$ , (qHu, Q5) => (none, qHu, Q5, {})
|  $\Lambda$ , (Q5, Q10) => (none, none, Q5, Q10)
| W1, (Q5, Q10) => (none, none, Q5, Q10)
| W2, (qHd, Q10) => (qHd, none, {}, Q10)
| W3, (qHu, Q5) => (none, qHu, Q5, {})
| W4, (qHd, qHu, Q5) => (qHd, qHu, Q5, {})
| K1, (Q5, Q10) => (none, none, Q5, Q10)
| K2, (qHd, qHu, Q10) => (qHd, qHu, {}, Q10)
```



```
| topYukawa, (qHu, Q10) => (qHu, none, {}, Q10)
| bottomYukawa, (qHd, Q5, Q10) => (qHd, none, Q5, Q10)
```

def toChargeProfile[\[source\]](#)

For a given potential term T , the charge profile associated with a collection of charges.

```
def toChargeProfile : (T : PotentialTerm) → Charges → T.ChargeProfile
| μ, (qHd, qHu, _, _) => (qHd, qHu)
| β, (_, qHu, Q5, _) => (qHu, Q5)
| Λ, (_, _, Q5, Q10) => (Q5, Q10)
| W1, (_, _, Q5, Q10) => (Q5, Q10)
| W2, (qHd, _, _, Q10) => (qHd, Q10)
| W3, (_, qHu, Q5, _) => (qHu, Q5)
| W4, (qHd, qHu, Q5, _) => (qHd, qHu, Q5)
| K1, (_, _, Q5, Q10) => (Q5, Q10)
| K2, (qHd, qHu, _, Q10) => (qHd, qHu, Q10)
| topYukawa, (qHu, _, _, Q10) => (qHu, Q10)
| bottomYukawa, (qHd, _, Q5, Q10) => (qHd, Q5, Q10)
```

lemma fromChargeProfile_toChargeProfile[\[source\]](#)

```
@[simp]
```

```
lemma fromChargeProfile_toChargeProfile (T : PotentialTerm) (cp : T.ChargeProfile) :
  toChargeProfile T (fromChargeProfile T cp) = cp
```

lemma fromChargeProfile_injective[\[source\]](#)

```
lemma fromChargeProfile_injective (T : PotentialTerm) :
  Function.Injective (fromChargeProfile T)
```

lemma toChargeProfile_surjective[\[source\]](#)

```
lemma toChargeProfile_surjective (T : PotentialTerm) :
  Function.Surjective (toChargeProfile T)
```

lemma toChargeProfile_empty[\[source\]](#)

```
@[simp]
```

```
lemma toChargeProfile_empty (T : PotentialTerm) :
  toChargeProfile T ∅ = ∅
```

lemma fromChargeProfile_empty[\[source\]](#)

```
@[simp]
```

```
lemma fromChargeProfile_empty (T : PotentialTerm) :
  fromChargeProfile T ∅ = ∅
```

lemma toChargeProfile_subset_of_subset[\[source\]](#)

```
lemma toChargeProfile_subset_of_subset (T : PotentialTerm) {x y : Charges} (h : x ≤ y) :
  toChargeProfile T x ≤ toChargeProfile T y
```

lemma fromChargeProfile_subset_of_subset[\[source\]](#)

```
lemma fromChargeProfile_subset_of_subset {T : PotentialTerm} {x y : T.ChargeProfile} (h : x ⊆ y) :
  fromChargeProfile T x ⊆ fromChargeProfile T y
```

lemma fromChargeProfile_subset_iff_subset[\[source\]](#)

```
@[simp]
lemma fromChargeProfile_subset_iff_subset {T : PotentialTerm} {x y : T.ChargeProfile} :
  fromChargeProfile T x ⊆ fromChargeProfile T y ↔ x ⊆ y
```

def offFinset[\[source\]](#)

Given $S5\ S10 : \text{Finset } \mathbb{Z}$ the finite set of charges associated with for which the 5-bar representation charges sit in $S5$ and the 10d representation charges sit in $S10$.

```
def offFinset (S5 S10 : Finset ℤ) : Finset Charges
```

lemma qHd_mem_offFinset[\[source\]](#)

```
lemma qHd_mem_offFinset (S5 S10 : Finset ℤ) (z : ℤ) (x2 : Option ℤ × Finset ℤ × Finset ℤ)
  (hsub : (some z, x2) ∈ offFinset S5 S10) :
  z ∈ S5
```

lemma qHu_mem_offFinset[\[source\]](#)

```
lemma qHu_mem_offFinset (S5 S10 : Finset ℤ) (z : ℤ) (x1 : Option ℤ) (x2 : Finset ℤ × Finset ℤ)
  (hsub : (x1, some z, x2) ∈ offFinset S5 S10) :
  z ∈ S5
```

lemma mem_offFinset_Q5_subset[\[source\]](#)

```
lemma mem_offFinset_Q5_subset (S5 S10 : Finset ℤ)
  {x : Charges} (hx : x ∈ offFinset S5 S10) :
  x.2.2.1 ⊆ S5
```

lemma mem_offFinset_Q10_subset[\[source\]](#)

```
lemma mem_offFinset_Q10_subset (S5 S10 : Finset ℤ)
  {x : Charges} (hx : x ∈ offFinset S5 S10) :
  x.2.2.2 ⊆ S10
```

lemma mem_offFinset_of_subset[\[source\]](#)

```
lemma mem_offFinset_of_subset (S5 S10 : Finset ℤ)
  {x y : Charges} (h : x ⊆ y) (hy : y ∈ offFinset S5 S10) :
  x ∈ offFinset S5 S10
```

lemma toChargeProfile_mem_offFinset_of_mem_offFinset[\[source\]](#)

```

lemma toChargeProfile_mem_ofFinset_of_mem_ofFinset (T : PotentialTerm)
  {x : Charges} (S5 S10 : Finset ℤ) (hx : x ∈ ofFinset S5 S10) :
  toChargeProfile T x ∈ ChargeProfile.ofFinset T S5 S10

```

lemma fromChargeProfile_mem_ofFinset_iff_mem_ofFinset

[\[source\]](#)

```

@[simp]
lemma fromChargeProfile_mem_ofFinset_iff_mem_ofFinset {T : PotentialTerm}
  {x : T.ChargeProfile} (S5 S10 : Finset ℤ) :
  fromChargeProfile T x ∈ ofFinset S5 S10 ↔ x ∈ ChargeProfile.ofFinset T S5 S10

```

def minimalSuperSet

[\[source\]](#)

Given a collection of charges x in $\text{ofFinset } S5 \ S10$, the minimal charges y in $\text{ofFinset } S5 \ S10$ which are a super sets of x .

```

def minimalSuperSet (S5 S10 : Finset ℤ) (x : Charges) : Finset Charges

```

lemma self_subset_mem_minimalSuperSet

[\[source\]](#)

```

lemma self_subset_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ minimalSuperSet S5 S10 x) : x ⊆ y

```

lemma self_not_mem_minimalSuperSet

[\[source\]](#)

```

@[simp]
lemma self_not_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x : Charges) :
  x ∉ minimalSuperSet S5 S10 x

```

lemma self_neq_mem_minimalSuperSet

[\[source\]](#)

```

lemma self_neq_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ minimalSuperSet S5 S10 x) : x ≠ y

```

lemma insert_Q5_mem_minimalSuperSet

[\[source\]](#)

```

lemma insert_Q5_mem_minimalSuperSet {S5 S10 : Finset ℤ} {x : Charges}
  (z : ℤ) (hz : z ∈ S5) (hznot : z ∉ x.2.2.1) :
  (x.1, x.2.1, insert z x.2.2.1, x.2.2.2) ∈ minimalSuperSet S5 S10 x

```

lemma insert_Q10_mem_minimalSuperSet

[\[source\]](#)

```

lemma insert_Q10_mem_minimalSuperSet {S5 S10 : Finset ℤ} {x : Charges}
  (z : ℤ) (hz : z ∈ S10) (hznot : z ∉ x.2.2.2) :
  (x.1, x.2.1, x.2.2.1, insert z x.2.2.2) ∈ minimalSuperSet S5 S10 x

```

lemma some_qHd_mem_minimalSuperSet_of_none

[\[source\]](#)

```

lemma some_qHd_mem_minimalSuperSet_of_none {S5 S10 : Finset ℤ} {x2 : Option ℤ × Finset ℤ ×
  Finset ℤ}
  (z : ℤ) (hz : z ∈ S5) :

```

```
(some z, x2) ∈ minimalSuperSet S5 S10 (none, x2)
```

lemma some_qHu_mem_minimalSuperSet_of_none

[\[source\]](#)

```
lemma some_qHu_mem_minimalSuperSet_of_none {S5 S10 : Finset ℤ}
  {x1 : Option ℤ} {x2 : Finset ℤ × Finset ℤ} (z : ℤ) (hz : z ∈ S5) :
  (x1, some z, x2) ∈ minimalSuperSet S5 S10 (x1, none, x2)
```

lemma exists_minimalSuperSet

[\[source\]](#)

```
lemma exists_minimalSuperSet (S5 S10 : Finset ℤ) {x y : Charges}
  (hy : y ∈ offFinset S5 S10) (hsubset : x ⊆ y)
  (hxneqy : x ≠ y) : ∃ z ∈ minimalSuperSet S5 S10 x, z ⊆ y
```

def IsComplete

[\[source\]](#)

A collection of charges is complete if it has all types of fields.

```
def IsComplete (x : Charges) : Prop
```

def completions

[\[source\]](#)

Given a collection of charges x in $\text{offFinset } S5 \ S10$, the minimal charges y in $\text{offFinset } S5 \ S10$ which are a super sets of x and are complete.

```
def completions (S5 S10 : Finset ℤ) (x : Charges) : Multiset Charges
```

lemma completions_eq_singleton_of_complete

[\[source\]](#)

```
lemma completions_eq_singleton_of_complete {S5 S10 : Finset ℤ} (x : Charges)
  (hcomplete : IsComplete x) :
  completions S5 S10 x = {x}
```

lemma self_mem_completions_iff_isComplete

[\[source\]](#)

```
@[simp]
lemma self_mem_completions_iff_isComplete {S5 S10 : Finset ℤ} (x : Charges) :
  x ∈ completions S5 S10 x ↔ IsComplete x
```

lemma mem_completions_isComplete

[\[source\]](#)

```
lemma mem_completions_isComplete {S5 S10 : Finset ℤ} {x y : Charges}
  (hx : y ∈ completions S5 S10 x) : IsComplete y
```

lemma self_subset_mem_completions

[\[source\]](#)

```
lemma self_subset_mem_completions (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ completions S5 S10 x) : x ⊆ y
```

lemma exist_completions_subset_of_complete[\[source\]](#)

```

lemma exist_completions_subset_of_complete (S5 S10 : Finset  $\mathbb{Z}$ ) (x y : Charges)
  (hsubset : x  $\subseteq$  y) (hy : y  $\in$  ofFinset S5 S10) (hycomplete : IsComplete y) :
   $\exists$  z  $\in$  completions S5 S10 x, z  $\subseteq$  y

```

2.17 PhysLean.StringTheory.FTheory.SU5U1.Charges.OfRationalSection

[PhysLean/StringTheory/FTheory/SU5U1/Charges/OfRationalSection.lean](#) on GitHub.

instance Fintype CodimensionOneConfig[\[source\]](#)

```

instance : Fintype CodimensionOneConfig where

```

def allowedBarFiveChargesList[\[source\]](#)

The allowed $U(1)$ -charges of matter in the 5-bar representation of $SU(5)$ given a CodimensionOneConfig, as ordered lists.

```

def allowedBarFiveChargesList : CodimensionOneConfig  $\rightarrow$  List  $\mathbb{Z}$ 
| same => [-3, -2, -1, 0, 1, 2, 3]
| nearestNeighbor => [-14, -9, -4, 1, 6, 11]
| nextToNearestNeighbor => [-13, -8, -3, 2, 7, 12]

```

lemma mem_allowedBarFiveChargesList_iff[\[source\]](#)

```

@[simp]
lemma mem_allowedBarFiveChargesList_iff {I : CodimensionOneConfig} (x :  $\mathbb{Z}$ ) :
  x  $\in$  I.allowedBarFiveChargesList  $\leftrightarrow$  x  $\in$  I.allowedBarFiveCharges

```

lemma allowedBarFiveChargesList_nodup[\[source\]](#)

```

lemma allowedBarFiveChargesList_nodup (I : CodimensionOneConfig) :
  I.allowedBarFiveChargesList.Nodup

```

def allowedTenChargesList[\[source\]](#)

The allowed $U(1)$ -charges of matter in the 10d representation of $SU(5)$ given a CodimensionOneConfig, as ordered lists.

```

def allowedTenChargesList : CodimensionOneConfig  $\rightarrow$  List  $\mathbb{Z}$ 
| same => [-3, -2, -1, 0, 1, 2, 3]
| nearestNeighbor => [-12, -7, -2, 3, 8, 13]
| nextToNearestNeighbor => [-9, -4, 1, 6, 11]

```

lemma mem_allowedTenChargesList_iff[\[source\]](#)

```

@[simp]
lemma mem_allowedTenChargesList_iff {I : CodimensionOneConfig} (x :  $\mathbb{Z}$ ) :
  x  $\in$  I.allowedTenChargesList  $\leftrightarrow$  x  $\in$  I.allowedTenCharges

```

lemma allowedTenChargesList_nodup[\[source\]](#)

```
lemma allowedTenChargesList_nodup (I : CodimensionOneConfig) :
  I.allowedTenChargesList.Nodup
```

def fiveChargeMultisetToList[\[source\]](#)

Given a multiset $S : \text{Multiset } \mathbb{Z}$ for which all elements are members of $I.\text{allowedBarFiveCharges}$, $\text{fiveChargeMultisetToList } I \ S$ is a computable list whose underlying multiset is S .

```
def fiveChargeMultisetToList (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ ) : List  $\mathbb{Z}$ 
```

def tenChargeMultisetToList[\[source\]](#)

Given a multiset $S : \text{Multiset } \mathbb{Z}$ for which all elements are members of $I.\text{allowedTenCharges}$, $\text{tenChargeMultisetToList } I \ S$ is a computable list whose underlying multiset is S .

```
def tenChargeMultisetToList (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ ) : List  $\mathbb{Z}$ 
```

lemma fiveChargeMultisetToList_mem_iff[\[source\]](#)

```
lemma fiveChargeMultisetToList_mem_iff {I : CodimensionOneConfig} {S : Multiset  $\mathbb{Z}$ } {a :  $\mathbb{Z}$ }:
  a  $\in$  fiveChargeMultisetToList I S  $\leftrightarrow$  (a  $\in$  S  $\wedge$  a  $\in$  I.allowedBarFiveCharges)
```

lemma tenChargeMultisetToList_mem_iff[\[source\]](#)

```
lemma tenChargeMultisetToList_mem_iff {I : CodimensionOneConfig} {S : Multiset  $\mathbb{Z}$ } {a :  $\mathbb{Z}$ }:
  a  $\in$  tenChargeMultisetToList I S  $\leftrightarrow$  (a  $\in$  S  $\wedge$  a  $\in$  I.allowedTenCharges)
```

lemma fiveChargeMultisetToList_count[\[source\]](#)

```
lemma fiveChargeMultisetToList_count {I : CodimensionOneConfig} {S : Multiset  $\mathbb{Z}$ } {a :  $\mathbb{Z}$ }
  (hmem : a  $\in$  I.allowedBarFiveCharges) :
  (fiveChargeMultisetToList I S).count a = S.count a
```

lemma tenChargeMultisetToList_count[\[source\]](#)

```
lemma tenChargeMultisetToList_count {I : CodimensionOneConfig} {S : Multiset  $\mathbb{Z}$ } {a :  $\mathbb{Z}$ }
  (hmem : a  $\in$  I.allowedTenCharges) :
  (tenChargeMultisetToList I S).count a = S.count a
```

lemma coe_fiveChargeMultisetToList_of_all_mem[\[source\]](#)

```
lemma coe_fiveChargeMultisetToList_of_all_mem (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ )
  (hs :  $\forall s \in S, s \in I.\text{allowedBarFiveCharges}$ ) :
   $\uparrow$ (fiveChargeMultisetToList I S) = S
```

lemma coe_tenChargeMultisetToList_of_all_mem[\[source\]](#)

```
lemma coe_tenChargeMultisetToList_of_all_mem (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ )
  (hs :  $\forall s \in S, s \in I.\text{allowedTenCharges}$ ) :
   $\uparrow$ (tenChargeMultisetToList I S) = S
```

2.18 PhysLean.StringTheory.FTheory.SU5U1.Fluxes.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Fluxes/Basic.lean](#) on GitHub.

def FluxesFive [\[source\]](#)

The fluxes (M, N) of the 5-bar matter curves of a theory which lead to chiral matter content in the MSSM.

```
def FluxesFive : Type
```

instance DecidableEq FluxesFive [\[source\]](#)

```
instance : DecidableEq FluxesFive
```

lemma zero_not_mem [\[source\]](#)

```
lemma zero_not_mem (F : FluxesFive) : (0, 0) ∉ F.1
```

def chiralIndicesOfD [\[source\]](#)

The multiset of chiral indices of the representation $D = (\bar{3}, 1)_{\{1/3\}}$ arising from the matter 5d representations.

```
def chiralIndicesOfD (F : FluxesFive) : Multiset ℤ
```

def numChiralD [\[source\]](#)

The total number of chiral D representations arising from the matter 5d representations.

```
def numChiralD (F : FluxesFive) : ℤ
```

def numAntiChiralD [\[source\]](#)

The total number of anti-chiral D representations arising from the matter 5d representations.

```
def numAntiChiralD (F : FluxesFive) : ℤ
```

def chiralIndicesOfL [\[source\]](#)

The multiset of chiral indices of the representation $L = (1, 2)_{\{-1/2\}}$ arising from the matter 5d representations.

```
def chiralIndicesOfL (F : FluxesFive) : Multiset ℤ
```

def numChiralL [\[source\]](#)

The total number of chiral L representations arising from the matter 5d representations.

```
def numChiralL (F : FluxesFive) : ℤ
```

def numAntiChiralL[\[source\]](#)

The total number of anti-chiral L representations arising from the matter 5d representations.

```
def numAntiChiralL (F : FluxesFive) : ℤ
```

def FluxesFive.NoExotics[\[source\]](#)

The condition that the 5d-matter representations do not lead to exotic chiral matter in the MSSM spectrum. This corresponds to the conditions that: - There are 3 chiral L representations and no anti-chiral L representations. - There are 3 chiral D representations and no anti-chiral D representations.

```
def NoExotics (F : FluxesFive) : Prop
```

instance Decidable (F.NoExotics)[\[source\]](#)

```
instance (F : FluxesFive) : Decidable (F.NoExotics)
```

def FluxesTen[\[source\]](#)

The fluxes (M, N) of the 10d matter curves of a theory which lead to chiral matter content in the MSSM.

```
def FluxesTen : Type
```

instance DecidableEq FluxesTen[\[source\]](#)

```
instance : DecidableEq FluxesTen
```

def chiralIndicesOfQ[\[source\]](#)

The multiset of chiral indices of the representation $Q = (3,2)_{1/6}$ arising from the matter 10d representations, corresponding to M.

```
def chiralIndicesOfQ (F : FluxesTen) : Multiset ℤ
```

def numChiralQ[\[source\]](#)

The total number of chiral Q representations arising from the matter 10d representations.

```
def numChiralQ (F : FluxesTen) : ℤ
```

def numAntiChiralQ[\[source\]](#)

The total number of anti-chiral Q representations arising from the matter 10d representations.

```
def numAntiChiralQ (F : FluxesTen) : ℤ
```


def chiralIndicesOfU[\[source\]](#)

The multiset of chiral indices of the representation $U = (\bar{3}, 1)_{-2/3}$ arising from the matter 10d representations, corresponding to $M - N$

```
def chiralIndicesOfU (F : FluxesTen) : Multiset ℤ
```

def numChiralU[\[source\]](#)

The total number of chiral U representations arising from the matter 10d representations.

```
def numChiralU (F : FluxesTen) : ℤ
```

def numAntiChiralU[\[source\]](#)

The total number of anti-chiral U representations arising from the matter 10d representations.

```
def numAntiChiralU (F : FluxesTen) : ℤ
```

def chiralIndicesOfE[\[source\]](#)

The multiset of chiral indices of the representation $E = (1, 1)_{1}$ arising from the matter 10d representations, corresponding to $M + N$

```
def chiralIndicesOfE (F : FluxesTen) : Multiset ℤ
```

def numChiralE[\[source\]](#)

The total number of chiral E representations arising from the matter 10d representations.

```
def numChiralE (F : FluxesTen) : ℤ
```

def numAntiChiralE[\[source\]](#)

The total number of anti-chiral E representations arising from the matter 10d representations.

```
def numAntiChiralE (F : FluxesTen) : ℤ
```

def FluxesTen.NoExotics[\[source\]](#)

The condition that the 10d-matter representations do not lead to exotic chiral matter in the MSSM spectrum. This corresponds to the conditions that: - There are 3 chiral Q representations and no anti-chiral Q representations. - There are 3 chiral U representations and no anti-chiral U representations. - There are 3 chiral E representations and no anti-chiral E representations.

```
def NoExotics (F : FluxesTen) : Prop
```

instance Decidable (F.NoExotics)

[\[source\]](#)

```
instance (F : FluxesTen) : Decidable (F.NoExotics)
```

2.19 PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.ChiralIndices

[PhysLean/StringTheory/FTheory/SU5U1/Fluxes/NoExotics/ChiralIndices.lean](#) on GitHub.

lemma chiralIndicesOfD_noneg_of_noExotics

[\[source\]](#)

The chiral indices of the representations $D = (\bar{3}, 1)_{1/3}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfD_noneg_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) : 0 ≤ ci
```

lemma chiralIndicesOfD_sum_eq_three_of_noExotics

[\[source\]](#)

The sum of the chiral indices of the representations $D = (\bar{3}, 1)_{1/3}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfD_sum_eq_three_of_noExotics (F : FluxesFive) (hF : NoExotics F) :
  F.chiralIndicesOfD.sum = 3
```

lemma chiralIndicesOfD_le_three_of_noExotics

[\[source\]](#)

The chiral indices of the representation $D = (\bar{3}, 1)_{1/3}$ are less then or equal to 3.

```
lemma chiralIndicesOfD_le_three_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) : ci ≤ 3
```

lemma mem_chiralIndicesOfD_mem_of_noExotics

[\[source\]](#)

```
lemma mem_chiralIndicesOfD_mem_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfD_subset_sum_le_three_of_noExotics

[\[source\]](#)

```
lemma chiralIndicesOfD_subset_sum_le_three_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F.1) : (S.map (fun x => x.1)).sum ≤ 3
```

lemma chiralIndicesOfL_noneg_of_noExotics

[\[source\]](#)

The chiral indices of the representations $L = (1, 2)_{-1/2}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfL_noneg_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) : 0 ≤ ci
```

lemma chiralIndicesOfL_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $L = (1,2)_{-1/2}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfL_sum_eq_three_of_noExotics (F : FluxesFive) (hF : NoExotics F) :
  F.chiralIndicesOfL.sum = 3
```

lemma chiralIndicesOfL_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $L = (1,2)_{-1/2}$ are less then or equal to 3.

```
lemma chiralIndicesOfL_le_three_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) : ci ≤ 3
```

lemma mem_chiralIndicesOfL_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfL_mem_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfL_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndicesOfL_subset_sum_le_three_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F.1) : (S.map (fun x => (x.1 + x.2))).sum ≤ 3
```

lemma chiralIndicesOfQ_noneg_of_noExotics[\[source\]](#)

The chiral indices of the representations $Q = (3,2)_{1/6}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfQ_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) : 0 ≤ ci
```

lemma chiralIndicesOfQ_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $Q = (3,2)_{1/6}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfQ_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndicesOfQ.sum = 3
```

lemma chiralIndicesOfQ_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $Q = (3,2)_{1/6}$ are less then or equal to 3.

```
lemma chiralIndicesOfQ_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) : ci ≤ 3
```

lemma mem_chiralIndicesOfQ_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfQ_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) :
```

$$ci \in (\{0, 1, 2, 3\} : \text{Finset } \mathbb{Z})$$
lemma chiralIndicesOfQ_subset_sum_le_three_of_noExotics
[\[source\]](#)

```
lemma chiralIndicesOfQ_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F.1) : (S.map (fun x => x.1)).sum ≤ 3
```

lemma chiralIndicesOfU_noneg_of_noExotics
[\[source\]](#)

The chiral indices of the representations $U = (\text{bar } 3, 1)_{-2/3}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfU_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfU) : 0 ≤ ci
```

lemma chiralIndicesOfU_sum_eq_three_of_noExotics
[\[source\]](#)

The sum of the chiral indices of the representations $U = (\text{bar } 3, 1)_{-2/3}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfU_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndicesOfU.sum = 3
```

lemma chiralIndicesOfU_le_three_of_noExotics
[\[source\]](#)

The chiral indices of the representation $U = (\text{bar } 3, 1)_{-2/3}$ are less then or equal to 3.

```
lemma chiralIndicesOfU_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfU) : ci ≤ 3
```

lemma mem_chiralIndicesOfU_mem_of_noExotics
[\[source\]](#)

```
lemma mem_chiralIndicesOfU_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfU) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfU_subset_sum_le_three_of_noExotics
[\[source\]](#)

```
lemma chiralIndicesOfU_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F.1) : (S.map (fun x => (x.1 - x.2))).sum ≤ 3
```

lemma chiralIndicesOfE_noneg_of_noExotics
[\[source\]](#)

```
lemma chiralIndicesOfE_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfE) : 0 ≤ ci
```

lemma chiralIndicesOfE_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $E = (1,1)_{\{1\}}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfE_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndicesOfE.sum = 3
```

lemma chiralIndicesOfE_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $E = (1,1)_{\{1\}}$ are less then or equal to 3.

```
lemma chiralIndicesOfE_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfE) : ci ≤ 3
```

lemma mem_chiralIndicesOfE_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfE_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfE) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfE_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndicesOfE_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F.1) : (S.map (fun x => (x.1 + x.2))).sum ≤ 3
```

2.20 PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.Completeness

[PhysLean/StringTheory/FTheory/SU5U1/Fluxes/NoExotics/Completeness.lean](#) on GitHub.

lemma FluxesFive.mem_mem_finset_of_noExotics[\[source\]](#)

```
lemma mem_mem_finset_of_noExotics (F : FluxesFive) (hF : F.NoExotics)
  (x : ℤ × ℤ) (hx : x ∈ F.1) :
  x ∈ ({(0, 1), (0, 2), (0, 3), (1, -1), (1, 0), (1, 1), (1, 2),
    (2, -2), (2, -1), (2, 0), (2, 1), (3, -3), (3, -2), (3, -1), (3, 0)} : Finset (ℤ × ℤ))
```

lemma FluxesFive.subset_le_mem_of_card_eq_one[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_one {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 1) (hS : S ≤ F.1) :
  S ∈ ({(0, 1), (0, 2), (0, 3), (1, -1), (1, 0), (1, 1), (1, 2),
    (2, -2), (2, -1), (2, 0), (2, 1), (3, -3), (3, -2), (3, -1), (3, 0)} :
    Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesFive.subset_le_mem_of_card_eq_two[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_two {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 2) (hS : S ≤ F.1) :
  S ∈ ({(0, 1), (0, 1), (0, 1), (1, -1), (0, 1), (1, 0), (0, 1), (0, 2),
    (0, 1), (1, 1), (0, 1), (2, -2), (0, 1), (2, -1), (0, 1), (2, 0),
    (0, 1), (3, -3), (0, 1), (3, -2), (0, 1), (3, -1),
    (0, 2), (1, -1), (0, 2), (1, 0), (0, 2), (2, -2), (0, 2), (2, -1),
```

```
{(0, 2), (3, -3)}, {(0, 2), (3, -2)}, {(0, 3), (1, -1)}, {(0, 3), (2, -2)},
{(0, 3), (3, -3)}, {(1, -1), (1, -1)}, {(1, -1), (1, 0)}, {(1, -1), (1, 1)},
{(1, -1), (1, 2)}, {(1, -1), (2, -2)}, {(1, -1), (2, -1)}, {(1, -1), (2, 0)},
{(1, 0), (1, 0)}, {(1, -1), (2, 1)}, {(1, 0), (1, 1)}, {(1, 0), (2, -2)},
{(1, 0), (2, -1)}, {(1, 0), (2, 0)}, {(1, 1), (2, -2)}, {(1, 1), (2, -1)},
{(1, 2), (2, -2)}} : Finset (Multiset ( $\mathbb{Z} \times \mathbb{Z}$ ))
```

lemma FluxesFive.subset_le_mem_of_card_eq_three

[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_three {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset ( $\mathbb{Z} \times \mathbb{Z}$ )) (hcard : S.card = 3) (hS : S ≤ F.1) :
  S ∈ ({{(0, 1), (0, 1), (0, 1)}, {(0, 1), (0, 1), (1, -1)},
    {(0, 1), (0, 1), (1, 0)}, {(0, 1), (0, 1), (2, -2)},
    {(0, 1), (0, 1), (2, -1)}, {(0, 1), (0, 1), (3, -3)}, {(0, 1), (0, 1), (3, -2)},
    {(0, 1), (0, 2), (1, -1)}, {(0, 1), (0, 2), (2, -2)}, {(0, 1), (0, 2), (3, -3)},
    {(0, 1), (1, -1), (1, -1)}, {(0, 1), (1, -1), (1, 0)}, {(0, 1), (1, -1), (1, 1)},
    {(0, 1), (1, -1), (2, -2)}, {(0, 1), (1, -1), (2, -1)}, {(0, 1), (1, -1), (2, 0)},
    {(0, 1), (1, 0), (1, 0)}, {(0, 1), (1, 0), (2, -2)}, {(0, 1), (1, 0), (2, -1)},
    {(0, 1), (1, 1), (2, -2)}, {(0, 2), (1, -1), (1, -1)}, {(0, 2), (1, -1), (1, 0)},
    {(0, 2), (1, -1), (2, -2)}, {(0, 2), (1, -1), (2, -1)}, {(0, 2), (1, 0), (2, -2)},
    {(0, 3), (1, -1), (1, -1)}, {(0, 3), (1, -1), (2, -2)}, {(1, -1), (1, -1), (1, -1)},
    {(1, -1), (1, -1), (1, 0)}, {(1, -1), (1, -1), (1, 1)}, {(1, -1), (1, -1), (1, 2)},
    {(1, -1), (1, 0), (1, 0)}, {(1, -1), (1, 0), (1, 1)}, {(1, 0), (1, 0), (1, 0)}} :
  Finset (Multiset ( $\mathbb{Z} \times \mathbb{Z}$ ))
```

lemma FluxesFive.subset_le_mem_of_card_eq_four

[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_four {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset ( $\mathbb{Z} \times \mathbb{Z}$ )) (hcard : S.card = 4) (hS : S ≤ F.1) :
  S ∈ ({{(0, 1), (0, 1), (0, 1), (1, -1)},
    {(0, 1), (0, 1), (0, 1), (2, -2)}, {(0, 1), (0, 1), (0, 1), (3, -3)},
    {(0, 1), (0, 1), (1, -1), (1, -1)}, {(0, 1), (0, 1), (1, -1), (1, 0)},
    {(0, 1), (0, 1), (1, -1), (2, -2)}, {(0, 1), (0, 1), (1, -1), (2, -1)},
    {(0, 1), (0, 1), (1, 0), (2, -2)}, {(0, 1), (0, 2), (1, -1), (1, -1)},
    {(0, 1), (0, 2), (1, -1), (2, -2)}, {(0, 1), (1, -1), (1, -1), (1, -1)},
    {(0, 1), (1, -1), (1, -1), (1, 0)}, {(0, 1), (1, -1), (1, -1), (1, 1)},
    {(0, 1), (1, -1), (1, 0), (1, 0)}, {(0, 2), (1, -1), (1, -1), (1, -1)},
    {(0, 2), (1, -1), (1, -1), (1, 0)}, {(0, 3), (1, -1), (1, -1), (1, -1)}} :
  Finset (Multiset ( $\mathbb{Z} \times \mathbb{Z}$ ))
```

lemma subset_le_mem_of_card_eq_five

[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_five {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset ( $\mathbb{Z} \times \mathbb{Z}$ )) (hcard : S.card = 5) (hS : S ≤ F.1) :
  S ∈ ({{(0, 1), (0, 1), (0, 1), (1, -1), (1, -1)},
    {(0, 1), (0, 1), (0, 1), (1, -1), (2, -2)},
    {(0, 1), (0, 1), (1, -1), (1, -1), (1, -1)},
    {(0, 1), (0, 1), (1, -1), (1, -1), (1, 0)},
    {(0, 1), (0, 2), (1, -1), (1, -1), (1, -1)}} :
  Finset (Multiset ( $\mathbb{Z} \times \mathbb{Z}$ ))
```

lemma subset_le_mem_of_card_eq_six

[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_six {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset ( $\mathbb{Z} \times \mathbb{Z}$ )) (hcard : S.card = 6) (hS : S ≤ F.1) :
```

```
S ∈ ({(0, 1), (0, 1), (0, 1), (1, -1), (1, -1), (1, -1)}) :
Finset (Multiset (ℤ × ℤ))
```

lemma subset_le_mem_of_card_eq_seven[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_seven {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 7) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))
```

lemma subset_le_mem_of_card_eq_seven_add[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_seven_add {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) {n : ℕ} (hcard : S.card = 7 + n) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))
```

lemma subset_le_mem_of_card_ge_seven[\[source\]](#)

```
lemma subset_le_mem_of_card_ge_seven {F : FluxesFive} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : 7 ≤ S.card) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))
```

lemma card_le_six_of_noExotics[\[source\]](#)

```
lemma card_le_six_of_noExotics {F : FluxesFive} (hF : F.NoExotics) :
  F.1.card ≤ 6
```

lemma card_mem_range_seven_of_noExotics[\[source\]](#)

```
lemma card_mem_range_seven_of_noExotics {F : FluxesFive} (hF : F.NoExotics) :
  F.1.card ∈ Finset.range 7
```

lemma FluxesFive.mem_elemsNoExotics_of_noExotics[\[source\]](#)

```
lemma mem_elemsNoExotics_of_noExotics (F : FluxesFive) (hNE : F.NoExotics) :
  F ∈ elemsNoExotics
```

lemma FluxesFive.noExotics_iff_mem_elemsNoExotics[\[source\]](#)

Completeness of elemsNoExotics, that is, every element of FluxesFive which obeys NoExotics is an element of elemsNoExotics, and every element of elemsNoExotics obeys NoExotics.

```
lemma noExotics_iff_mem_elemsNoExotics (F : FluxesFive) :
  F.NoExotics ↔ F ∈ elemsNoExotics
```

lemma FluxesTen.mem_mem_finset_of_noExotics[\[source\]](#)

```
lemma mem_mem_finset_of_noExotics (F : FluxesTen) (hF : F.NoExotics)
  (x : ℤ × ℤ) (hx : x ∈ F.1) :
  x ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (ℤ × ℤ))
```

lemma FluxesTen.subset_le_mem_of_card_eq_one[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_one {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 1) (hS : S ≤ F.1) :
  S ∈ ({(1, -1)}, {(1, 0)}, {(1, 1)}, {(2, -1)}, {(2, 0)}, {(2, 1)}, {(3, 0)}} :
  Finset (Multiset (ℤ × ℤ)))

```

lemma FluxesTen.subset_le_mem_of_card_eq_two[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_two {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 2) (hS : S ≤ F.1) :
  S ∈ ({(1, -1), (1, 0)}, {(1, -1), (1, 1)}, {(1, -1), (2, 1)},
    {(1, 0), (1, 0)}, {(1, 0), (1, 1)}, {(1, 0), (2, 0)}, {(1, 1), (2, -1)}} :
  Finset (Multiset (ℤ × ℤ)))

```

lemma FluxesTen.subset_le_mem_of_card_eq_three[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_three {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 3) (hS : S ≤ F.1) :
  S ∈ ({(1, -1), (1, 0), (1, 1)}, {(1, 0), (1, 0), (1, 0)}} : Finset (Multiset (ℤ × ℤ)))

```

lemma FluxesTen.subset_le_mem_of_card_eq_four[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_four {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 4) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma subset_le_mem_of_card_eq_four_add[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_four_add {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) {n : ℕ} (hcard : S.card = 4 + n) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma subset_le_mem_of_card_ge_four[\[source\]](#)

```

lemma subset_le_mem_of_card_ge_four {F : FluxesTen} (hF : F.NoExotics)
  (S : Multiset (ℤ × ℤ)) (hcard : 4 ≤ S.card) (hS : S ≤ F.1) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma card_le_three_of_noExotics[\[source\]](#)

```

lemma card_le_three_of_noExotics {F : FluxesTen} (hF : F.NoExotics) :
  F.1.card ≤ 3

```

lemma card_mem_range_four_of_noExotics[\[source\]](#)

```

lemma card_mem_range_four_of_noExotics {F : FluxesTen} (hF : F.NoExotics) :
  F.1.card ∈ Finset.range 4

```

lemma FluxesTen.mem_elemsNoExotics_of_noExotics[\[source\]](#)


```
lemma mem_elemsNoExotics_of_noExotics (F : FluxesTen) (hNE : F.NoExotics) :
  F ∈ elemsNoExotics
```

lemma FluxesTen.noExotics_iff_mem_elemsNoExotics

[\[source\]](#)

Completeness of elemsNoExotics, that is, every element of FluxesTen which obeys NoExotics is an element of elemsNoExotics, and every element of elemsNoExotics obeys NoExotics.

```
lemma noExotics_iff_mem_elemsNoExotics (F : FluxesTen) :
  F.NoExotics ↔ F ∈ elemsNoExotics
```

2.21 PhysLean.StringTheory.FTheory.SU5U1.Fluxes.NoExotics.Elems

[PhysLean/StringTheory/FTheory/SU5U1/Fluxes/NoExotics/Elems.lean](#) on GitHub.

def FluxesFive.elemsNoExotics

[\[source\]](#)

The elements of FluxesFive for which the NoExotics condition holds.

```
def elemsNoExotics : Multiset FluxesFive
```

lemma FluxesFive.elemsNoExotics_card

[\[source\]](#)

```
lemma elemsNoExotics_card : elemsNoExotics.card = 31
```

lemma FluxesFive.elemsNoExotics_nodup

[\[source\]](#)

```
lemma elemsNoExotics_nodup : elemsNoExotics.Nodup
```

lemma FluxesFive.noExotics_of_mem_elemsNoExotics

[\[source\]](#)

```
lemma noExotics_of_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  NoExotics F
```

lemma toFinset_card_le_four_mem_elemsNoExotics

[\[source\]](#)

```
lemma toFinset_card_le_four_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  F.l.toFinset.card ≤ 4
```

lemma FluxesFive.sum_of_mem_elemsNoExotics

[\[source\]](#)

```
lemma sum_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :
  F.l.sum = (3, 0)
```

def FluxesTen.elemsNoExotics

[\[source\]](#)

The elements of FluxesTen for which the NoExotics condition holds.

```
def elemsNoExotics : Multiset FluxesTen
```

lemma FluxesTen.elemsNoExotics_card[\[source\]](#)

```
lemma elemsNoExotics_card : elemsNoExotics.card = 6
```

lemma FluxesTen.elemsNoExotics_nodup[\[source\]](#)

```
lemma elemsNoExotics_nodup : elemsNoExotics.Nodup
```

lemma FluxesTen.noExotics_of_mem_elemsNoExotics[\[source\]](#)

```
lemma noExotics_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :  
  NoExotics F
```

lemma toFinset_card_le_three_mem_elemsNoExotics[\[source\]](#)

```
lemma toFinset_card_le_three_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :  
  F.1.toFinset.card ≤ 3
```

lemma FluxesTen.sum_of_mem_elemsNoExotics[\[source\]](#)

```
lemma sum_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :  
  F.1.sum = (3, 0)
```

2.22 PhysLean.StringTheory.FTheory.SU5U1.Matter

[PhysLean/StringTheory/FTheory/SU5U1/Matter.lean](#) on GitHub.

abbrev QuantaBarFive.MN[\[source\]](#)

The ChiralityFlux and HyperChargeFlux quanta of a 5-bar representation.

```
abbrev QuantaBarFive.MN (a : QuantaBarFive) : ChiralityFlux × HyperChargeFlux
```

abbrev QuantaTen.MN[\[source\]](#)

The ChiralityFlux and HyperChargeFlux quanta of a 10d representation.

```
abbrev QuantaTen.MN (a : QuantaTen) : ChiralityFlux × HyperChargeFlux
```

instance Decidable (DistinctChargedBarFive quantaBarFiveMatter qHu qHd)[\[source\]](#)

```
instance (quantaBarFiveMatter : Multiset (QuantaBarFive)) (qHu : ℤ) (qHd : ℤ) :  
  Decidable (DistinctChargedBarFive quantaBarFiveMatter qHu qHd)
```

instance Decidable (DistinctChargedTen quantaTen)[\[source\]](#)

```
instance (quantaTen : Multiset QuantaTen) :  
  Decidable (DistinctChargedTen quantaTen)
```

lemma quantaBarFiveMatter_map_MN_not_both_zero[\[source\]](#)

lemma quantaBarFiveMatter_map_MN_not_both_zero :
 $\forall a \in (\mathcal{M}.\text{quantaBarFiveMatter}.\text{map } \text{QuantaBarFive.MN}), (a.1 = 0 \rightarrow a.2 \neq 0)$

abbrev Q5[\[source\]](#)

The multiset of charges associated with the 5-bar matter content.

abbrev Q5 : Multiset $\mathbb{Z}$

lemma Q5_def[\[source\]](#)

lemma Q5_def : $\mathcal{M}.Q5 = \mathcal{M}.\text{quantaBarFiveMatter}.\text{map } (\text{QuantaBarFive}.q)$

lemma Q5_subset_allowedBarFiveCharges[\[source\]](#)

lemma Q5_subset_allowedBarFiveCharges : $\mathcal{M}.Q5.\text{toFinset} \subseteq I.\text{allowedBarFiveCharges}$

lemma Q5_noDup[\[source\]](#)

lemma Q5_noDup : $\mathcal{M}.Q5.\text{Nodup}$

lemma Q5_eq_toFinset[\[source\]](#)

lemma Q5_eq_toFinset : $\mathcal{M}.Q5 = \mathcal{M}.Q5.\text{toFinset}.1$

lemma Q5_mem_powerset[\[source\]](#)

lemma Q5_mem_powerset : $\mathcal{M}.Q5.\text{toFinset} \in I.\text{allowedBarFiveCharges}.\text{powerset}$

lemma Q5_mem_powerset_filter_card[\[source\]](#)

lemma Q5_mem_powerset_filter_card {n : $\mathbb{N}$ }
 (hcard : $\mathcal{M}.\text{quantaBarFiveMatter}.\text{card} = n$) : $\mathcal{M}.Q5.\text{toFinset} \in$
 $I.\text{allowedBarFiveCharges}.\text{powerset}.\text{filter } \text{fun } x \Rightarrow x.\text{card} = n$

lemma quantaBarFive_map_q_subset_allowedBarFiveCharges[\[source\]](#)

lemma quantaBarFive_map_q_subset_allowedBarFiveCharges :
 $(\mathcal{M}.\text{quantaBarFive}.\text{map } (\text{QuantaBarFive}.q)).\text{toFinset} \subseteq I.\text{allowedBarFiveCharges}$

lemma quantaBarFive_map_q_mem_powerset[\[source\]](#)

lemma quantaBarFive_map_q_mem_powerset :
 $(\mathcal{M}.\text{quantaBarFive}.\text{map } (\text{QuantaBarFive}.q)).\text{toFinset} \in I.\text{allowedBarFiveCharges}.\text{powerset}$

abbrev Q10[\[source\]](#)

The multiset of charges associated with the 10d matter content.

abbrev Q10 : Multiset $\mathbb{Z}$

lemma Q10_def [\[source\]](#)

lemma Q10_def : $\mathcal{M}.Q10 = \mathcal{M}.quantaTen.map \text{QuantaTen}.q$

lemma Q10_subset_allowedTenCharges [\[source\]](#)

lemma Q10_subset_allowedTenCharges : $\mathcal{M}.Q10.toFinset \subseteq I.allowedTenCharges$

lemma Q10_nodup [\[source\]](#)

lemma Q10_nodup : $\mathcal{M}.Q10.Nodup$

lemma Q10_eq_toFinset [\[source\]](#)

lemma Q10_eq_toFinset : $\mathcal{M}.Q10 = \mathcal{M}.Q10.toFinset.1$

lemma Q10_mem_powerset [\[source\]](#)

lemma Q10_mem_powerset : $\mathcal{M}.Q10.toFinset \in I.allowedTenCharges.powerset$

2.23 PhysLean.StringTheory.FTheory.SU5U1.Potential.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Potential/Basic.lean](#) on GitHub.

inductive PotentialTerm [\[source\]](#)

Relevant terms part of the superpotential and Kahler potential of the SU(5) SUSY GUT.

inductive PotentialTerm

```
| μ : PotentialTerm
| β : PotentialTerm
| Λ : PotentialTerm
| W1 : PotentialTerm
| W2 : PotentialTerm
| W3 : PotentialTerm
| W4 : PotentialTerm
| K1 : PotentialTerm
| K2 : PotentialTerm
| topYukawa : PotentialTerm
| bottomYukawa : PotentialTerm
```

def violateRParity [\[source\]](#)

The finite set of terms in the superpotential and Kahler potential which violate R-parity. $-\beta_i 5^{\sim M^i} 5Hu - \lambda_{ijk} 5^{\sim M^i} 5^{\sim M^j} 10^k - W^2_{ijk} 10^i 10^j 10^k 5^{\sim Hd} - W^4_i 5^{\sim M^i} 5^{\sim Hd} 5Hu 5Hu - K^1_{ijk} 10^i 10^j 5M^k - K^2_i 5^{\sim Hu} 5^{\sim Hd} 10^i$ These correspond to the terms with an odd number of matter fields.

def violateRParity : Finset PotentialTerm

def causeProtonDecay[\[source\]](#)

The finite set of terms in the superpotential and Kahler potential which are involved in proton decay. $-W^1_{ijk} 10^i 10^j 10^k 5^{-M^1} - \lambda_{ijk} 5^{-M^i} 5^{-M^j} 10^k - W^2_{ijk} 10^i 10^j 10^k 5^{-Hd} - K^1_{ijk} 10^i 10^j 5M^k$

```
def causeProtonDecay : Finset PotentialTerm
```

2.24 PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Potential/ChargeProfile/Basic.lean](#) on GitHub.

def ChargeProfile[\[source\]](#)

The charge profiles of the potential terms in the $SU(5) \times U(1)$ theory. This assigns to each potential term T a type $T.ChargeProfile$ which specifies the possible values of charges one can assign to the fields present in T .

The types correspond to: $-\mu : qHd \times qHu - \beta : qHu \times Q5 - \Lambda : Q5 \times Q10 - W1 : Q5 \times Q10 - W2 : qHd \times Q10 - W3 : qHu \times Q5 - W4 : qHd \times qHu \times Q5 - K1 : Q5 \times Q10 - K2 : qHd \times qHu \times Q10 - topYukawa : qHu \times Q10 - bottomYukawa : qHd \times Q5 \times Q10$

```
def ChargeProfile : PotentialTerm → Type
```

```
instance (T : PotentialTerm) → DecidableEq T.ChargeProfile | μ => inferInstanceAs...
```

[\[source\]](#)

For each term in the potential the charge profile is a decidable type.

```
instance : (T : PotentialTerm) → DecidableEq T.ChargeProfile
| μ => inferInstanceAs (DecidableEq (Option ℤ × Option ℤ))
| β => inferInstanceAs (DecidableEq (Option ℤ × Finset ℤ))
| Λ => inferInstanceAs (DecidableEq (Finset ℤ × Finset ℤ))
| W1 => inferInstanceAs (DecidableEq (Finset ℤ × Finset ℤ))
| W2 => inferInstanceAs (DecidableEq (Option ℤ × Finset ℤ))
| W3 => inferInstanceAs (DecidableEq (Option ℤ × Finset ℤ))
| W4 => inferInstanceAs (DecidableEq (Option ℤ × Option ℤ × Finset ℤ))
| K1 => inferInstanceAs (DecidableEq (Finset ℤ × Finset ℤ))
| K2 => inferInstanceAs (DecidableEq (Option ℤ × Option ℤ × Finset ℤ))
| topYukawa => inferInstanceAs (DecidableEq (Option ℤ × Finset ℤ))
| bottomYukawa => inferInstanceAs (DecidableEq (Option ℤ × Finset ℤ × Finset ℤ))
```

instance HasSubset T.ChargeProfile[\[source\]](#)

```
instance (T : PotentialTerm) : HasSubset T.ChargeProfile where Subset x y
```

instance subsetDecidable[\[source\]](#)

```
instance subsetDecidable : (T : PotentialTerm) → (x y : T.ChargeProfile) → Decidable (x ≤ y)
| μ, _, _ => instDecidableAnd
| β, _, _ => instDecidableAnd
| Λ, _, _ => instDecidableAnd
| W1, _, _ => instDecidableAnd
| W2, _, _ => instDecidableAnd
| W3, _, _ => instDecidableAnd
| W4, _, _ => instDecidableAnd
| K1, _, _ => instDecidableAnd
```

```
| K2, _, _ => instDecidableAnd
| topYukawa, _, _ => instDecidableAnd
| bottomYukawa, _, _ => instDecidableAnd
```

lemma subset_refl[\[source\]](#)

```
@[simp, refl]
```

```
lemma subset_refl {T : PotentialTerm} (x : T.ChargeProfile) : x ⊆ x
```

lemma subset_trans[\[source\]](#)

```
@[trans]
```

```
lemma subset_trans {T : PotentialTerm} {x y z : T.ChargeProfile} (h1 : x ⊆ y) (h2 : y ⊆ z) :
  x ⊆ z
```

lemma toFinset_inj[\[source\]](#)

```
lemma _root_.Option.toFinset_inj {x y : Option ℤ} :
  x = y ↔ x.toFinset = y.toFinset
```

lemma subset_antisymm[\[source\]](#)

```
lemma subset_antisymm {T : PotentialTerm} {x y : T.ChargeProfile} (h1 : x ⊆ y) (h2 : y ⊆ x) :
  x = y
```

instance emptyInst[\[source\]](#)

```
instance emptyInst (T : PotentialTerm) : EmptyCollection T.ChargeProfile where
```

lemma empty_subset[\[source\]](#)

```
@[simp]
```

```
lemma empty_subset {T : PotentialTerm} (x : T.ChargeProfile) :
  ∅ ⊆ x
```

lemma subset_of_empty_iff_empty[\[source\]](#)

```
@[simp]
```

```
lemma subset_of_empty_iff_empty {T : PotentialTerm} {x : T.ChargeProfile} :
  x ⊆ ∅ ↔ x = ∅
```

def Option.powerset[\[source\]](#)

The powerset of $x : \text{Option } \mathbb{Z}$ defined as $\{\text{none}\}$ if x is none and $\{\text{none}, \text{some } y\}$ if x is some y .

```
def _root_.Option.powerset (x : Option ℤ) : Finset (Option ℤ)
```

lemma mem_powerset_iff[\[source\]](#)

```
@[simp]
lemma _root_.Option.mem_powerset_iff {x : Option Z} (y : Option Z) :
  y ∈ x.powerset ↔ y.toFinset ⊆ x.toFinset
```

def ChargeProfile.powerset[\[source\]](#)

The powerset of a charge profile. Given a charge profile $x : T.ChargeProfile$ it's powerset is the finite set of all $T.ChargeProfile$ which are subsets of x .

```
def powerset {T : PotentialTerm} (x : T.ChargeProfile) : Finset T.ChargeProfile
```

lemma mem_powerset_iff_subset[\[source\]](#)

```
@[simp]
lemma mem_powerset_iff_subset {T : PotentialTerm} {x y : T.ChargeProfile} :
  x ∈ powerset y ↔ x ⊆ y
```

lemma self_mem_powerset[\[source\]](#)

```
lemma self_mem_powerset {T : PotentialTerm} (x : T.ChargeProfile) :
  x ∈ powerset x
```

lemma empty_mem_powerset[\[source\]](#)

```
lemma empty_mem_powerset {T : PotentialTerm} (x : T.ChargeProfile) :
  ∅ ∈ powerset x
```

lemma powerset_of_empty[\[source\]](#)

```
@[simp]
lemma powerset_of_empty {T : PotentialTerm} :
  powerset (∅ : T.ChargeProfile) = {∅}
```

lemma powerset_subset_iff_subset[\[source\]](#)

```
lemma powerset_subset_iff_subset {T : PotentialTerm} {x y : T.ChargeProfile} :
  powerset x ⊆ powerset y ↔ x ⊆ y
```

lemma min_exists_inductive[\[source\]](#)

```
lemma min_exists_inductive {T : PotentialTerm} (S : Finset T.ChargeProfile) (hS : S ≠ ∅) :
  (n : ℕ) → (hn : S.card = n) →
  ∃ y ∈ S, powerset y n S = {y}
| 0, h => by simp_all
| 1, h => by
  rw [Finset.card_eq_one] at h
  obtain ⟨y, rfl⟩
```

lemma min_exists[\[source\]](#)

```
lemma min_exists {T : PotentialTerm} (S : Finset T.ChargeProfile) (hS : S ≠ ∅) :
  ∃ y ∈ S, powerset y n S = {y}
```

instance PartialOrder T.ChargeProfile[\[source\]](#)

```
instance {T : PotentialTerm} : PartialOrder T.ChargeProfile where
```

def charges[\[source\]](#)

The $U(1)$ charges of each potential term given an element of the corresponding ChargeProfile. For example, for the term $\beta_i 5^{\sim M^i} 5Hu$ and $Q5 = \{0, 2\}$ and $qHu = 3$ then the charges of this term would be $\{-3, -1\}$.

```
def charges : {T : PotentialTerm} → T.ChargeProfile → Multiset ℤ
| μ, (qHd, qHu) => (qHd.toFinset.product <| qHu.toFinset).val.map (fun x => x.1 - x.2)
| β, (qHu, Q5) => (qHu.toFinset.product <| Q5).val.map (fun x => - x.1 + x.2)
| Λ, (Q5, Q10) => (Q5.product <| Q5.product <| Q10).val.map (fun x => x.1 + x.2.1 + x.2.2)
| W1, (Q5, Q10) => (Q5.product <| Q10.product <| Q10.product <| Q10).val.map
  (fun x => x.1 + x.2.1 + x.2.2.1 + x.2.2.2)
| W2, (qHd, Q10) => (qHd.toFinset.product <| Q10.product <| Q10.product <| Q10).val.map
  (fun x => x.1 + x.2.1 + x.2.2.1 + x.2.2.2)
| W3, (qHu, Q5) => (qHu.toFinset.product <| Q5.product <| Q5).val.map
  (fun x => -x.1 - x.1 + x.2.1 + x.2.2)
| W4, (qHd, qHu, Q5) => (qHd.toFinset.product <| qHu.toFinset.product <| Q5).val.map
  (fun x => x.1 - x.2.1 - x.2.1 + x.2.2)
| K1, (Q5, Q10) => (Q5.product <| Q10.product <| Q10).val.map
  (fun x => - x.1 + x.2.1 + x.2.2)
| K2, (qHd, qHu, Q10) => (qHd.toFinset.product <| qHu.toFinset.product <| Q10).val.map
  (fun x => x.1 + x.2.1 + x.2.2)
| topYukawa, (qHu, Q10) => (qHu.toFinset.product <| Q10.product <| Q10).val.map
  (fun x => -x.1 + x.2.1 + x.2.2)
| bottomYukawa, (qHd, Q5, Q10) => (qHd.toFinset.product <| Q5.product <| Q10).val.map
  (fun x => x.1 + x.2.1 + x.2.2)
```

lemma charges_of_subset[\[source\]](#)

```
lemma charges_of_subset {T : PotentialTerm} {x y : T.ChargeProfile} (h : x ⊆ y) :
  charges x ⊆ charges y
```

def IsExcluded[\[source\]](#)

Given a potential term T with an element of it's associated ChargeProfile, the proposition corresponding to there been no zero in the charges of that term. That is to say, it is excluded by the $U(1)$ charges.

For example, for the term $\beta_i 5^{\sim M^i} 5Hu$ and $Q5 = \{0, 2\}$ and $qHu = 3$ the charges of this term are $\{-3, -1\}$, and this is excluded by the $U(1)$ charges. Thus $IsExcluded \beta (3, \{0, 2\})$ would be true.

```
def IsExcluded {T : PotentialTerm} (x : T.ChargeProfile) : Prop
```


instance (T : PotentialTerm) → DecidablePred (IsExcluded (T

[\[source\]](#)

The decidability of IsExcluded for the potential terms.

```
instance : (T : PotentialTerm) → DecidablePred (IsExcluded (T := T))
| μ => fun _ => instDecidableNot
| β => fun _ => instDecidableNot
| Λ => fun _ => instDecidableNot
| W1 => fun _ => instDecidableNot
| W2 => fun _ => instDecidableNot
| W3 => fun _ => instDecidableNot
| W4 => fun _ => instDecidableNot
| K1 => fun _ => instDecidableNot
| K2 => fun _ => instDecidableNot
| topYukawa => fun _ => instDecidableNot
| bottomYukawa => fun _ => instDecidableNot
```

def IsPresent

[\[source\]](#)

Given a potential term T with an element of it's associated ChargeProfile, the proposition corresponding to there been zero in the charges of that term. That is to say, it is present by the $U(1)$ charges.

For example, for the term $\beta_1 5 \bar{M}^5 H_u$ and $Q5 = \{0, 2\}$ and $qHu = 2$ the charges of this term are $\{-2, 0\}$, and this is excluded by the $U(1)$ charges. Thus $\text{IsPresent } \beta (3, \{0, 2\})$ would be true.

```
def IsPresent (T : PotentialTerm) (x : T.ChargeProfile) : Prop
```

lemma not_isExcluded_iff_isPresent

[\[source\]](#)

```
lemma not_isExcluded_iff_isPresent : (T : PotentialTerm) → (x : T.ChargeProfile) →
  ¬ x.IsExcluded ↔ x.IsPresent
| μ => fun (qHd, qHu) => by simp [IsExcluded, IsPresent]
| β => fun (qHu, Q5) => by simp [IsExcluded, IsPresent]
| Λ => fun (Q5, Q10) => by simp [IsExcluded, IsPresent]
| W1 => fun (Q5, Q10) => by simp [IsExcluded, IsPresent]
| W2 => fun (qHd, Q10) => by simp [IsExcluded, IsPresent]
| W3 => fun (qHu, Q5) => by simp [IsExcluded, IsPresent]
| W4 => fun (qHd, qHu, Q5) => by simp [IsExcluded, IsPresent]
| K1 => fun (Q5, Q10) => by simp [IsExcluded, IsPresent]
| K2 => fun (qHd, qHu, Q10) => by simp [IsExcluded, IsPresent]
| topYukawa => fun (qHu, Q10) => by simp [IsExcluded, IsPresent]
| bottomYukawa => fun (qHd, Q5, Q10) => by simp [IsExcluded, IsPresent]
```

instance DecidablePred (IsPresent (T

[\[source\]](#)

The decidability of IsPresent for the potential terms.

```
instance (T : PotentialTerm) : DecidablePred (IsPresent (T := T))
```

lemma isPresent_of_subset

[\[source\]](#)

```
lemma isPresent_of_subset {T : PotentialTerm} {y x : T.ChargeProfile}
  (h : y ⊆ x) (hy : y.IsPresent) : x.IsPresent
```

def ofFinset[\[source\]](#)

Given $S5 \ S10 : \text{Finset } \mathbb{Z}$ the finite set of charge profiles associated with a potential term T for which the 5-bar representation charges sit in $S5$ and the 10d representation charges sit in $S10$.

```
def ofFinset (T : PotentialTerm) (S5 S10 : Finset  $\mathbb{Z}$ ) : Finset T.ChargeProfile
```

lemma mem_ofFinset_of_subset[\[source\]](#)

```
lemma mem_ofFinset_of_subset {T : PotentialTerm} (S5 S10 : Finset  $\mathbb{Z}$ )
  {x y : T.ChargeProfile} (h : x  $\subseteq$  y) (hy : y  $\in$  ofFinset T S5 S10) :
  x  $\in$  ofFinset T S5 S10
```

def finsetOfCodimensionOneConfig[\[source\]](#)

Given a $I : \text{CodimensionOneConfig}$, and a potential term PotentialTerm , the finite set of elements of $T.\text{ChargeProfile}$ which originate from charges allowed by I .

```
def finsetOfCodimensionOneConfig (I : CodimensionOneConfig) (T : PotentialTerm) :
  Finset T.ChargeProfile
```

2.25 PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Potential/ChargeProfile/Irreducible/Basic.lean](#) on GitHub.

def IsIrreducible[\[source\]](#)

For a potential term T , an $x : T.\text{ChargeProfile}$ is said to be irreducible if it allows the term T , and no other member of its powerset allows T .

```
def IsIrreducible {T : PotentialTerm} (x : T.ChargeProfile) : Prop
```

instance Decidable x.IsIrreducible[\[source\]](#)

```
instance {T : PotentialTerm} (x : T.ChargeProfile) : Decidable x.IsIrreducible
```

lemma isPresent_of_isIrreducible[\[source\]](#)

```
lemma isPresent_of_isIrreducible {T : PotentialTerm} {x : T.ChargeProfile} (h : IsIrreducible x) :
  IsPresent T x
```

lemma isPresent_of_has_isIrreducible_subset[\[source\]](#)

```
lemma isPresent_of_has_isIrreducible_subset {T : PotentialTerm} {x : T.ChargeProfile}
  (hx :  $\exists y \in \text{powerset } x, \text{IsIrreducible } y$ ) : IsPresent T x
```

lemma isIrreducible_iff_powerset_filter_eq[\[source\]](#)

```
lemma isIrreducible_iff_powerset_filter_eq {T : PotentialTerm} {x : T.ChargeProfile} :
  x.IsIrreducible  $\leftrightarrow$  x.powerset.filter (IsPresent T) = {x}
```

lemma isIrreducible_iff_powerset_countP_eq_one[\[source\]](#)

```
lemma isIrreducible_iff_powerset_countP_eq_one {T : PotentialTerm} {x : T.ChargeProfile} :
  x.IsIrreducible ↔ x.powerset.val.countP (IsPresent T) = 1
```

lemma subset_isIrreducible_of_isPresent[\[source\]](#)

```
lemma subset_isIrreducible_of_isPresent {T : PotentialTerm} {x : T.ChargeProfile}
  (hx : IsPresent T x) : ∃ y ∈ powerset x, IsIrreducible y
```

lemma isPresent_iff_subest_isIrreducible[\[source\]](#)

```
lemma isPresent_iff_subest_isIrreducible {T : PotentialTerm} {x : T.ChargeProfile} :
  IsPresent T x ↔ ∃ y ∈ powerset x, IsIrreducible y
```

2.26 PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Completeness

[PhysLean/StringTheory/FTheory/SU5U1/Potential/ChargeProfile/Irreducible/Completeness.lean](#) on GitHub.

def toMultisetsOne[\[source\]](#)

The multisets of cardinality 1 containing elements from a finite set s .

```
def toMultisetsOne (s : Finset ℤ) : Multiset (Multiset ℤ)
```

lemma mem_toMultisetsOne_iff[\[source\]](#)

```
@[simp]
lemma mem_toMultisetsOne_iff {s : Finset ℤ} (X : Multiset ℤ) :
  X ∈ toMultisetsOne s ↔ X.toFinset ⊆ s ∧ X.card = 1
```

def toMultisetsTwo[\[source\]](#)

The multisets of cardinality 2 containing elements from a finite set s .

```
def toMultisetsTwo (s : Finset ℤ) : Multiset (Multiset ℤ)
```

lemma mem_toMultisetsTwo_iff[\[source\]](#)

```
@[simp]
lemma mem_toMultisetsTwo_iff {s : Finset ℤ} (X : Multiset ℤ) :
  X ∈ toMultisetsTwo s ↔ X.toFinset ⊆ s ∧ X.card = 2
```

def toMultisetsThree[\[source\]](#)

The multisets of cardinality 3 containing elements from a finite set s .

```
def toMultisetsThree (s : Finset ℤ) : Multiset (Multiset ℤ)
```

lemma mem_toMultisetsThree_iff[\[source\]](#)

```
@[simp]
lemma mem_toMultisetsThree_iff {s : Finset ℤ} (X : Multiset ℤ) :
  X ∈ toMultisetsThree s ↔ X.toFinset ⊆ s ∧ X.card = 3
```

def irreducibleElems'[\[source\]](#)

For a given $I : \text{CodimensionOneConfig}$ and $T : \text{PotentialTerm}$, a Multiset of $T.\text{ChargeProfile}$. This multiset is equivalent to `irreducibleElem I T`, except defined based on more general subsets making it easier to use for proofs.

```
def irreducibleElems' (I : CodimensionOneConfig) : (T : PotentialTerm) → Multiset T.
  ChargeProfile
| μ =>
  let SqHd
```

lemma mem_irreducibleElems'_of_irreducible[\[source\]](#)

Elements of `irreducibleElems'` are irreducible.

```
lemma mem_irreducibleElems'_of_irreducible {I : CodimensionOneConfig} {T : PotentialTerm}
  (x : ChargeProfile T) (h : IsIrreducible x) (hx : x ∈ finsetOfCodimensionOneConfig I T) :
  x ∈ irreducibleElems' I T
```

lemma irreducibleElems'_card_eq[\[source\]](#)

```
lemma irreducibleElems'_card_eq (I : CodimensionOneConfig) (T : PotentialTerm) :
  (irreducibleElems' I T).card = irreducibleElemsCard I T
```

lemma irreducibleElems_eq_irreducibleElems'[\[source\]](#)

```
lemma irreducibleElems_eq_irreducibleElems' (I : CodimensionOneConfig) (T : PotentialTerm) :
  irreducibleElems I T = irreducibleElems' I T
```

lemma isIrreducible_iff_mem_irreducibleElems[\[source\]](#)

```
lemma isIrreducible_iff_mem_irreducibleElems {I : CodimensionOneConfig} {T : PotentialTerm}
  (x : ChargeProfile T) (hx : x ∈ finsetOfCodimensionOneConfig I T) :
  IsIrreducible x ↔ x ∈ irreducibleElems I T
```

2.27 PhysLean.StringTheory.FTheory.SU5U1.Potential.ChargeProfile.Irreducible.Elems

[PhysLean/StringTheory/FTheory/SU5U1/Potential/ChargeProfile/Irreducible/Elems.lean](#) on GitHub.

instance OfNat (Option ℤ) n[\[source\]](#)

OfNat instance on Option ℤ.

```
local instance (n : ℕ) : OfNat (Option ℤ) n where
```

instance Neg (Option $\mathbb{Z}$)[\[source\]](#)*Negation on Option $\mathbb{Z}$.***local instance** : Neg (Option $\mathbb{Z}$) **where****def irreducibleElemsOfSame**[\[source\]](#)*Irreducible charge profiles for when CodimensionOneConfig = same.*

```

def irreducibleElemsOfSame : (T : PotentialTerm) → Multiset T.ChargeProfile
| μ => {(-3, -3), (-2, -2), (-1, -1), (0, 0), (1, 1), (2, 2), (3, 3)}
| K2 => {(-3, 0, {3}), (-3, 1, {2}), (-3, 2, {1}), (-3, 3, {0}), (-2, -1, {3}),
        (-2, 0, {2}), (-2, 1, {1}), (-2, 2, {0}), (-2, 3, {-1}), (-1, -2, {3}), (-1, -1, {2}),
        (-1, 0, {1}), (-1, 1, {0}), (-1, 2, {-1}), (-1, 3, {-2}), (0, -3, {3}), (0, -2, {2}),
        (0, -1, {1}), (0, 0, {0}), (0, 1, {-1}), (0, 2, {-2}), (0, 3, {-3}), (1, -3, {2}),
        (1, -2, {1}), (1, -1, {0}), (1, 0, {-1}), (1, 1, {-2}), (1, 2, {-3}), (2, -3, {1}),
        (2, -2, {0}), (2, -1, {-1}), (2, 0, {-2}), (2, 1, {-3}), (3, -3, {0}), (3, -2, {-1}),
        (3, -1, {-2}), (3, 0, {-3})}
| K1 => {({-3}, {-2, -1}), ({-3}, {-3, 0}), ({-2}, {-1}), ({-2}, {-2, 0}), ({-2}, {-3, 1}),
        ({-1}, {-1, 0}), ({-1}, {-2, 1}), ({-1}, {-3, 2}), ({0}, {0}), ({0}, {-1, 1}), ({0}, {-2, 2}),
        ({0}, {-3, 3}), ({1}, {0, 1}), ({1}, {-1, 2}), ({1}, {-2, 3}), ({2}, {1}), ({2}, {0, 2}),
        ({2}, {-1, 3}), ({3}, {1, 2}), ({3}, {0, 3})}
| W4 => {(-3, -3, {-3}), (-3, -2, {-1}), (-3, -1, {1}), (-3, 0, {3}), (-2, -2, {-2}),
        (-2, -1, {0}), (-2, 0, {2}), (-1, -2, {-3}), (-1, -1, {-1}), (-1, 0, {1}), (-1, 1, {3}),
        (0, -1, {-2}), (0, 0, {0}), (0, 1, {2}), (1, -1, {-3}), (1, 0, {-1}), (1, 1, {1}), (1, 2, {3}),
        (2, 0, {-2}), (2, 1, {0}), (2, 2, {2}), (3, 0, {-3}), (3, 1, {-1}), (3, 2, {1}), (3, 3, {3})}
| W3 => {(-3, {-3}), (-2, {-2}), (-2, {-3, -1}), (-1, {-1}), (-1, {-2, 0}), (-1, {-3, 1}),
        (0, {0}), (0, {-1, 1}), (0, {-2, 2}), (0, {-3, 3}), (1, {1}), (1, {0, 2}), (1, {-1, 3}),
        (2, {2}), (2, {1, 3}), (3, {3})}
| W2 => {(-3, {1}), (-3, {-1, 2}), (-3, {-3, 3}), (-3, {0, 3}), (-3, {-2, 2, 3}),
        (-2, {0, 1}), (-2, {-2, 2}), (-2, {0, 2}), (-2, {-1, 1, 2}), (-2, {-1, 0, 3}),
        (-2, {-2, 1, 3}), (-2, {-3, 2, 3}), (-1, {-1, 1}), (-1, {0, 1}), (-1, {-3, 2}),
        (-1, {-1, 0, 2}), (-1, {-2, 1, 2}), (-1, {-1, 3}), (-1, {-2, 0, 3}), (-1, {-3, 1, 3}),
        (0, {0}), (0, {-2, 1}), (0, {-1, 2}), (0, {-3, 1, 2}), (0, {-2, -1, 3}), (1, {-1, 0}),
        (1, {-3, 1}), (1, {-1, 1}), (1, {-2, 0, 1}), (1, {-2, -1, 2}), (1, {-3, 0, 2}), (1, {-2, 3}),
        (1, {-3, -1, 3}), (2, {-2, 0}), (2, {-1, 0}), (2, {-2, -1, 1}), (2, {-3, 0, 1}), (2, {-2, 2}),
        (2, {-3, -1, 2}), (2, {-3, -2, 3}), (3, {-1}), (3, {-3, 0}), (3, {-2, 1}), (3, {-3, -2, 2}),
        (3, {-3, 3})}
| W1 => {({-3}, {1}), ({-3}, {-1, 2}), ({-3}, {-3, 3}), ({-3}, {0, 3}), ({-3}, {-2, 2, 3}),
        ({-2}, {0, 1}), ({-2}, {-2, 2}), ({-2}, {0, 2}), ({-2}, {-1, 1, 2}), ({-2}, {-1, 0, 3}),
        ({-2}, {-2, 1, 3}), ({-2}, {-3, 2, 3}), ({-1}, {-1, 1}), ({-1}, {0, 1}), ({-1}, {-3, 2}),
        ({-1}, {-1, 0, 2}), ({-1}, {-2, 1, 2}), ({-1}, {-1, 3}), ({-1}, {-2, 0, 3}), ({-1}, {-3, 1, 3}),
        ({0}, {0}), ({0}, {-2, 1}), ({0}, {-1, 2}), ({0}, {-3, 1, 2}), ({0}, {-2, -1, 3}),
        ({1}, {-1, 0}), ({1}, {-3, 1}), ({1}, {-1, 1}), ({1}, {-2, 0, 1}), ({1}, {-2, -1, 2}),
        ({1}, {-3, 0, 2}), ({1}, {-2, 3}), ({1}, {-3, -1, 3}), ({2}, {-2, 0}), ({2}, {-1, 0}),
        ({2}, {-2, -1, 1}), ({2}, {-3, 0, 1}), ({2}, {-2, 2}), ({2}, {-3, -1, 2}), ({2}, {-3, -2, 3}),
        ({3}, {-1}), ({3}, {-3, 0}), ({3}, {-2, 1}), ({3}, {-3, -2, 2}), ({3}, {-3, 3})}
| Λ => {({-1}, {2}), ({-2, -1}, {3}), ({0}, {0}), ({-3, 0}, {3}), ({-2, 0}, {2}),
        ({-1, 0}, {1}), ({1}, {-2}), ({-3, 1}, {2}), ({-2, 1}, {1}), ({-1, 1}, {0}), ({0, 1}, {-1})}

```

,

def irreducibleElemsOfNN[\[source\]](#)*Irreducible charge profiles for when CodimensionOneConfig = nearestNeighbor.*

```

def irreducibleElemsOfNN : (T : PotentialTerm) → Multiset T.ChargeProfile
| μ => {(-14, -14), (-9, -9), (-4, -4), (1, 1), (6, 6), (11, 11)}
| K2 => {(-14, 1, {13}), (-14, 6, {8}), (-14, 11, {3}), (-9, -4, {13}),
        (-9, 1, {8}), (-9, 6, {3}), (-9, 11, {-2}), (-4, -9, {13}), (-4, -4, {8}), (-4, 1, {3}),
        (-4, 6, {-2}), (-4, 11, {-7}), (1, -14, {13}), (1, -9, {8}), (1, -4, {3}), (1, 1, {-2}),
        (1, 6, {-7}), (1, 11, {-12}), (6, -14, {8}), (6, -9, {3}), (6, -4, {-2}), (6, 1, {-7}),
        (6, 6, {-12}), (11, -14, {3}), (11, -9, {-2}), (11, -4, {-7}), (11, 1, {-12})}
| K1 => {({-14}), {-7}), ({-14}, {-12, -2}), ({-9}, {-7, -2}), ({-9}, {-12, 3}),
        ({-4}, {-2}), ({-4}, {-7, 3}), ({-4}, {-12, 8}), ({1}, {-2, 3}), ({1}, {-7, 8}),
        ({1}, {-12, 13}), ({6}, {3}), ({6}, {-2, 8}), ({6}, {-7, 13}), ({11}, {3, 8}),
        ({11}, {-2, 13})}
| W4 => {(-14, -14, {-14}), (-14, -9, {-4}), (-14, -4, {6}), (-9, -9, {-9}),
        (-9, -4, {1}), (-9, 1, {11}), (-4, -9, {-14}), (-4, -4, {-4}), (-4, 1, {6}), (1, -4, {-9}),
        (1, 1, {1}), (1, 6, {11}), (6, -4, {-14}), (6, 1, {-4}), (6, 6, {6}), (11, 1, {-9}),
        (11, 6, {1}), (11, 11, {11})}
| W3 => {(-14, {-14}), (-9, {-9}), (-9, {-14, -4}), (-4, {-4}), (-4, {-9, 1}),
        (-4, {-14, 6}), (1, {1}), (1, {-4, 6}), (1, {-9, 11}), (6, {6}), (6, {1, 11}), (11, {11})}
| W2 => {(-14, {-2, 8}), (-14, {3, 8}), (-14, {-12, 13}), (-14, {-2, 3, 13}),
        (-14, {-7, 8, 13}), (-9, {3}), (-9, {-7, 8}), (-9, {-2, 13}), (-9, {-12, 8, 13}), (-4, {-2,
        3}),
        (-4, {-12, 8}), (-4, {-2, 8}), (-4, {-7, 3, 8}), (-4, {-7, -2, 13}), (-4, {-12, 3, 13}),
        (1, {-7, 3}), (1, {-2, 3}), (1, {-7, -2, 8}), (1, {-12, 3, 8}), (1, {-7, 13}),
        (1, {-12, -2, 13}), (6, {-2}), (6, {-12, 3}), (6, {-7, 8}), (6, {-12, -7, 13}), (11, {-7,
        -2}),
        (11, {-7, 3}), (11, {-12, -2, 3}), (11, {-12, -7, 8}), (11, {-12, 13})}
| W1 => {({-14}, {-2, 8}), ({-14}, {3, 8}), ({-14}, {-12, 13}),
        ({-14}, {-2, 3, 13}), ({-14}, {-7, 8, 13}), ({-9}, {3}), ({-9}, {-7, 8}), ({-9}, {-2, 13}),
        ({-9}, {-12, 8, 13}), ({-4}, {-2, 3}), ({-4}, {-12, 8}), ({-4}, {-2, 8}), ({-4}, {-7, 3,
        8}),
        ({-4}, {-7, -2, 13}), ({-4}, {-12, 3, 13}), ({1}, {-7, 3}), ({1}, {-2, 3}), ({1}, {-7, -2,
        8}),
        ({1}, {-12, 3, 8}), ({1}, {-7, 13}), ({1}, {-12, -2, 13}), ({6}, {-2}), ({6}, {-12, 3}),
        ({6}, {-7, 8}), ({6}, {-12, -7, 13}), ({11}, {-7, -2}), ({11}, {-7, 3}), ({11}, {-12, -2,
        3}),
        ({11}, {-12, -7, 8}), ({11}, {-12, 13})}
| Λ => {({-4}, {8}), ({-9, -4}, {13}), ({1}, {-2}), ({-14, 1}, {13}),
        ({-9, 1}, {8}), ({-4, 1}, {3}), ({6}, {-12}), ({-14, 6}, {8}), ({-9, 6}, {3}), ({-4, 6},
        {-2}),
        ({1, 6}, {-7}), ({-14, 11}, {3}), ({-9, 11}, {-2}), ({-4, 11}, {-7}), ({1, 11}, {-12})}
| β => {(-14, {-14}), (-9, {-9}), (-4, {-4}), (1, {1}), (6, {6}), (11, {11})}
| topYukawa => {(-14, {-7}), (-14, {-12, -2}), (-9, {-7, -2}), (-9, {-12, 3}),
        (-4, {-2}), (-4, {-7, 3}), (-4, {-12, 8}), (1, {-2, 3}), (1, {-7, 8}), (1, {-12, 13}), (6,
        {3}),
        (6, {-2, 8}), (6, {-7, 13}), (11, {3, 8}), (11, {-2, 13})}
| bottomYukawa => {(-14, {1}, {13}), (-14, {6}, {8}), (-14, {11}, {3}),
        (-9, {-4}, {13}), (-9, {1}, {8}), (-9, {6}, {3}), (-9, {11}, {-2}), (-4, {-9}, {13}),
        (-4, {-4}, {8}), (-4, {1}, {3}), (-4, {6}, {-2}), (-4, {11}, {-7}), (1, {-14}, {13}),

```

def irreducibleElemsOfNTonN[\[source\]](#)*Irreducible charge profiles for when CodimensionOneConfig = nextToNearestNeighbor.*

```

def irreducibleElemsOfNToNN : (T : PotentialTerm) → Multiset T.ChargeProfile
| μ => {(-13, -13), (-8, -8), (-3, -3), (2, 2), (7, 7), (12, 12)}
| K2 => {(-13, 2, {11}), (-13, 7, {6}), (-13, 12, {1}), (-8, -3, {11}),
        (-8, 2, {6}), (-8, 7, {1}), (-8, 12, {-4}), (-3, -8, {11}), (-3, -3, {6}), (-3, 2, {1}),
        (-3, 7, {-4}), (-3, 12, {-9}), (2, -13, {11}), (2, -8, {6}), (2, -3, {1}), (2, 2, {-4}),
        (2, 7, {-9}), (7, -13, {6}), (7, -8, {1}), (7, -3, {-4}), (7, 2, {-9}), (12, -13, {1}),
        (12, -8, {-4}), (12, -3, {-9})}
| K1 => {({-13}, {-9, -4}), ({-8}, {-4}), ({-8}, {-9, 1}),
        ({-3}, {-4, 1}), ({-3}, {-9, 6}), ({2}, {1}), ({2}, {-4, 6}), ({2}, {-9, 11}), ({7}, {1,
        6}),
        ({7}, {-4, 11}), ({12}, {6}), ({12}, {1, 11})}
| W4 => {(-13, -13, {-13}), (-13, -8, {-3}), (-13, -3, {7}),
        (-8, -8, {-8}), (-8, -3, {2}), (-8, 2, {12}), (-3, -8, {-13}), (-3, -3, {-3}), (-3, 2, {7})
        ,
        (2, -3, {-8}), (2, 2, {2}), (2, 7, {12}), (7, -3, {-13}), (7, 2, {-3}), (7, 7, {7}),
        (12, 2, {-8}), (12, 7, {2}), (12, 12, {12})}
| W3 => {(-13, {-13}), (-8, {-8}), (-8, {-13, -3}), (-3, {-3}),
        (-3, {-8, 2}), (-3, {-13, 7}), (2, {2}), (2, {-3, 7}), (2, {-8, 12}), (7, {7}),
        (7, {2, 12}), (12, {12})}
| W2 => {(-13, {1, 6}), (-13, {-9, 11}), (-13, {1, 11}),
        (-13, {-4, 6, 11}), (-8, {-4, 6}), (-8, {1, 6}), (-8, {-4, 1, 11}), (-8, {-9, 6, 11}),
        (-3, {1}), (-3, {-9, 6}), (-3, {-4, 11}), (2, {-4, 1}), (2, {-4, 6}), (2, {-9, 1, 6}),
        (2, {-9, -4, 11}), (7, {-9, 1}), (7, {-4, 1}), (7, {-9, -4, 6}), (7, {-9, 11}), (12, {-4}),
        (12, {-9, 6})}
| W1 => {({-13}, {1, 6}), ({-13}, {-9, 11}), ({-13}, {1, 11}),
        ({-13}, {-4, 6, 11}), ({-8}, {-4, 6}), ({-8}, {1, 6}), ({-8}, {-4, 1, 11}), ({-8}, {-9, 6,
        11}),
        ({-3}, {1}), ({-3}, {-9, 6}), ({-3}, {-4, 11}), ({2}, {-4, 1}), ({2}, {-4, 6}),
        ({2}, {-9, 1, 6}), ({2}, {-9, -4, 11}), ({7}, {-9, 1}), ({7}, {-4, 1}), ({7}, {-9, -4, 6}),
        ({7}, {-9, 11}), ({12}, {-4}), ({12}, {-9, 6})}
| Λ => {({-3}, {6}), ({-8, -3}, {11}), ({2}, {-4}), ({-13, 2}, {11}),
        ({-8, 2}, {6}), ({-3, 2}, {1}), ({-13, 7}, {6}), ({-8, 7}, {1}), ({-3, 7}, {-4}),
        ({2, 7}, {-9}), ({-13, 12}, {1}), ({-8, 12}, {-4}), ({-3, 12}, {-9})}
| β => {(-13, {-13}), (-8, {-8}), (-3, {-3}), (2, {2}), (7, {7}),
        (12, {12})}
| topYukawa => {(-13, {-9, -4}), (-8, {-4}), (-8, {-9, 1}), (-3, {-4, 1}),
        (-3, {-9, 6}), (2, {1}), (2, {-4, 6}), (2, {-9, 11}), (7, {1, 6}), (7, {-4, 11}), (12, {6})
        ,
        (12, {1, 11})}
| bottomYukawa => {(-13, {2}, {11}), (-13, {7}, {6}), (-13, {12}, {1}),
        (-8, {-3}, {11}), (-8, {2}, {6}), (-8, {7}, {1}), (-8, {12}, {-4}), (-3, {-8}, {11}),
        (-3, {-3}, {6}), (-3, {2}, {1}), (-3, {7}, {-4}), (-3, {12}, {-9}), (2, {-13}, {11}),
        (2, {-8}, {6}), (2, {-3}, {1}), (2, {2}, {-4}), (2, {7}, {-9}), (7, {-13}, {6}), (7, {-8},
        {1}),
        (7, {-3}, {-4}), (7, {2}, {-9}), (12, {-13}, {1}), (12, {-8}, {-4}), (12, {-3}, {-9})}

```

def irreducibleElems

[\[source\]](#)

Irreducible charge profiles for a given CodimensionOneConfig.

```

def irreducibleElems: (I : CodimensionOneConfig) → (T : PotentialTerm) → Multiset T.
  ChargeProfile
| same, T => irreducibleElemsOfSame T
| nearestNeighbor, T => irreducibleElemsOfNN T
| nextToNearestNeighbor, T => irreducibleElemsOfNToNN T

```

lemma irreducibleElems_countP_not_isIrreducible[\[source\]](#)

```

lemma irreducibleElems_countP_not_isIrreducible
  (I : CodimensionOneConfig) (T : PotentialTerm) :
  ((irreducibleElems I T).countP (fun x => ¬ x.IsIrreducible)) = 0

```

lemma isIrreducible_of_mem_irreducibleElems[\[source\]](#)

```

lemma isIrreducible_of_mem_irreducibleElems
  {I : CodimensionOneConfig} {T : PotentialTerm} (x : T.ChargeProfile)
  (h : x ∈ irreducibleElems I T) : IsIrreducible x

```

lemma irreducibleElems_nodup[\[source\]](#)

```

lemma irreducibleElems_nodup (I : CodimensionOneConfig) (T : PotentialTerm) :
  (irreducibleElems I T).Nodup

```

lemma isPresent_of_mem_irreducibleElems[\[source\]](#)

```

lemma isPresent_of_mem_irreducibleElems
  {I : CodimensionOneConfig} {T : PotentialTerm} (x : T.ChargeProfile)
  (h : x ∈ irreducibleElems I T) : IsPresent T x

```

lemma irreducibleElems_subset_finsetOfCodimensionOneConfig[\[source\]](#)

```

lemma irreducibleElems_subset_finsetOfCodimensionOneConfig
  (I : CodimensionOneConfig) (T : PotentialTerm) :
  irreducibleElems I T ⊆ (finsetOfCodimensionOneConfig I T).val

```

def irreducibleElemsCard[\[source\]](#)

For $I : \text{CodimensionOneConfig}$ and $T : \text{PotentialTerm}$, the cardinality of $\text{irreducibleElems } I \ T$.

```

def irreducibleElemsCard (I : CodimensionOneConfig) (T : PotentialTerm) : ℕ

```

lemma irreducibleElems_card_eq[\[source\]](#)

```

lemma irreducibleElems_card_eq (I : CodimensionOneConfig) (T : PotentialTerm) :
  (irreducibleElems I T).card = irreducibleElemsCard I T

```